

AI Agents, Tool Calling & Protocols

Reasoning Patterns, Tool Calling, MCP, Memory, Durable Orchestration, Multi-Agent Systems & Production Safety

Target Persona: Senior AI Engineer / Applied AI Engineer (~3 YOE)

Focus Areas: ReAct, tool-schema design, MCP security, Context/State/Memory, durable execution, multi-agent coordination, agent evaluation, indirect prompt injection

Includes: Step-by-Step Numerical Hand Calculations, Python Implementations, SVG/HTML Diagrams

Module 01: Agent Fundamentals & Reasoning Patterns

1. Introduction & Intuition

The Core Bottleneck

A fixed pipeline — call the model once, maybe retrieve something first, generate an answer, done — has a hardcoded control flow: the sequence of steps is decided by the *developer*, at design time, not by the *model*, at run time. That's fine for tasks whose steps are genuinely fixed and known in advance. It breaks down the moment the right sequence of steps actually depends on what happens along the way — the right next action depends on what the *previous* action returned, and that can't be known until the model actually sees it. An agent is what you get when you hand that step-by-step control flow decision to the model itself: instead of "always do A then B then C," the model decides, at each step, whether to act, what to act with, and whether the result means it's done or needs to try something else. That flexibility is genuinely powerful — and genuinely more expensive, slower, and harder to predict than a fixed pipeline, which is exactly why *when* to reach for an agent at all is this module's central question, not just *how* one works.

High-Level Intuition

A fixed pipeline is a recipe: do these steps, in this order, every time, regardless of how the dish is turning out. An agent is a cook: taste as you go, decide whether it needs more salt, and only move to plating once it actually tastes right — the *sequence* of actions is decided dynamically, in response to what's actually observed, not fixed in advance. This is powerful precisely when you can't write the recipe in advance because the right next step genuinely depends on information you don't have until a previous step returns it. It's needless overhead when the recipe was always going to be the same regardless of what happened along the way — nobody needs a cook improvising a dish that only ever has one correct sequence of steps.

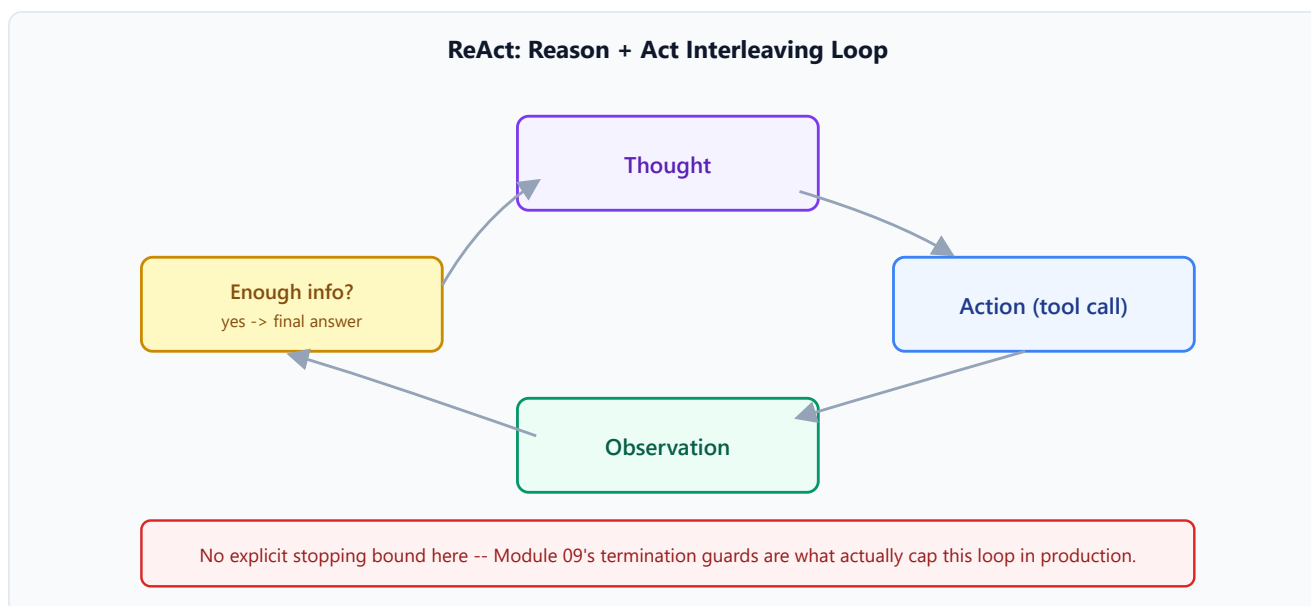
2. Core Concepts & Mathematical Formulation

This module is architectural throughout — reasoning patterns and the agent-vs-pipeline decision are procedural/comparative concepts, not closed-form calculations, consistent with how `03_advanced_rag` treated its own purely-architectural modules (Query Transformation, GraphRAG). What matters for interview readiness is the *mechanism* and, critically, *when each pattern is actually the right choice* — not a formula to memorize.

ReAct: Reason + Act Interleaving

Intuition & Practical Use

ReAct is the foundational agentic reasoning pattern: interleave explicit reasoning ("Thought") with concrete actions ("Action") and their results ("Observation"), in a loop, until the model decides it has enough information to answer. The model doesn't just silently decide what tool to call — it first reasons in text about *why*, which serves two real purposes: it measurably improves the *quality* of the model's action choices (reasoning before acting reduces impulsive, poorly-justified tool calls), and it makes the agent's behavior *inspectable* — a human debugging a bad run can read the Thought steps and see exactly where the reasoning went wrong, not just guess from the final answer.



Chain-of-Thought vs. Agentic Reasoning

Intuition & Practical Use

Chain-of-Thought (CoT) is reasoning *within* a single generation — the model writes out intermediate reasoning steps before its final answer, but it's still one call, with no ability to go fetch new information partway through. Agentic reasoning (ReAct and beyond) is reasoning *across* multiple calls, each informed by real, new information (a tool's actual output) the model didn't have when it started. The distinction matters: CoT can only ever reorganize and reason over what the model already knows or was given up front; an agent can reason, then go find out something it didn't know, then reason again with that new fact in hand. CoT reduces reasoning errors on a fixed set of information; agentic reasoning is what's actually needed when the task requires information the model doesn't have until it looks.

Plan-and-Execute vs. Purely Reactive Agents

Intuition & Practical Use

A purely reactive agent (like the ReAct loop above) decides its next single action based only on what it's seen so far — it never commits to a full plan up front, which makes it flexible but means it can wander: re-deciding its approach after every single observation, sometimes losing track of the original goal across many steps. A plan-and-execute agent instead first produces an explicit multi-step plan, *then* executes each step (potentially still reactively within a step), checking back against the plan rather than re-deciding the whole approach from scratch every time. The trade-off is real: an upfront plan gives structure and makes long-horizon tasks less likely to wander, but a plan committed to too early can be wrong in ways a purely reactive agent would have naturally adapted around — the right choice depends on how *predictable* the task's structure is upfront versus how much it depends on information only discoverable along the way.

When NOT to Use an Agent: A Formal Decision Framework

Intuition & Practical Use

This is the single highest-leverage question in this entire topic, and it's worth answering with a structured comparison rather than intuition alone. Four real architectural options exist, in increasing order of flexibility *and* cost/complexity/unpredictability — the right choice is the *least* flexible option that still genuinely satisfies the task, not the most sophisticated one available:

Architecture	Complexity	Cost	Latency	Reliability	Controllability	Right When
Single LLM call	Lowest	Lowest — one call	Lowest — one round trip	Highest — no compounding steps to fail	Highest — fully predictable input/output	The task is answerable from one prompt with no external information or multi-step action needed
Deterministic workflow	Low-Medium	Low — fixed, known number of calls	Predictable — fixed number of steps	High — each step is independently testable	High — the sequence is fixed by the developer, not the model	The steps and their order are genuinely knowable in advance, even if the content varies
Single agent	Medium-High	Variable, often higher — unbounded step count without a guard	Variable and unpredictable	Lower — errors can compound across self-directed steps	Lower — the model decides the sequence, not the developer	The right sequence of steps genuinely can't be known until runtime, but one coherent line of reasoning suffices

Architecture	Complexity	Cost	Latency	Reliability	Controllability	Right When
Multi-agent system	Highest	Highest — cost multiplies across agents	Highest and least predictable	Lowest — new, cross-agent failure modes appear (Module 06)	Lowest — coordination adds another layer of non-determinism	The task genuinely decomposes into specialized sub-problems that benefit from separate, focused agents — not merely because it "sounds complex"

The real discipline is climbing this table only as far as the task's genuine requirements force you to — a task solvable by a deterministic workflow gains nothing from being built as an agent except more ways to fail, more cost, and less predictability. (`03_advanced_rag` Module 08's Agentic RAG is the retrieval-specific instance of exactly this same table: a single agent reached for specifically when a fixed retrieve-then-generate pass demonstrably isn't good enough, not by default.)

3. Implementation & Reference Code

Below is a minimal, self-contained ReAct-style loop skeleton — illustrating the Thought → Action → Observation mechanics and the plan-and-execute vs. purely-reactive distinction, not a production framework. Real tool-calling mechanics (schema, parallel execution, idempotency) are Module 02's subject.

```

from dataclasses import dataclass, field
from enum import Enum

class StepType(Enum):
    THOUGHT = "thought"
    ACTION = "action"
    OBSERVATION = "observation"
    FINAL_ANSWER = "final_answer"

@dataclass
class Step:
    step_type: StepType
    content: str

@dataclass
class ReactTrace:
    """Full Thought/Action/Observation history for one agent run -- the inspectable

```

```

record ReAct's whole value proposition depends on."""
steps: list[Step] = field(default_factory=list)

def log(self, step_type: StepType, content: str) -> None:
    self.steps.append(Step(step_type, content))

def react_loop(query: str, reason_fn, act_fn, is_sufficient_fn, max_steps: int = 5) -> tuple[str,
ReactTrace]:
    """A minimal ReAct loop: Thought -> Action -> Observation, repeated until
    is_sufficient_fn says the model has enough information, or max_steps is hit.
    max_steps is a real, hard-coded termination guard -- never left implicit."""
    trace = ReactTrace()
    context = query

    for _ in range(max_steps):
        thought = reason_fn(context)
        trace.log(StepType.THUGHT, thought)

        if is_sufficient_fn(thought):
            trace.log(StepType.FINAL_ANSWER, thought)
            return thought, trace

        action = act_fn(thought)
        trace.log(StepType.ACTION, action)

        observation = f"[real tool execution result for: {action}]"
        trace.log(StepType.OBSERVATION, observation)
        context = f"{context}\nThought: {thought}\nAction: {action}\nObservation: {observation}"

        trace.log(StepType.FINAL_ANSWER, "[terminated: max_steps reached without sufficient
information]")
    return "[terminated: max_steps reached]", trace

def plan_and_execute(query: str, plan_fn, execute_step_fn) -> tuple[list[str], ReactTrace]:
    """Plan-and-execute: commit to a full plan upfront, then execute each step,
    contrasted against react_loop's step-by-step re-deciding above."""
    trace = ReactTrace()
    plan = plan_fn(query)
    trace.log(StepType.THUGHT, f"Committed plan: {plan}")

    results = []
    for step in plan:
        result = execute_step_fn(step)
        trace.log(StepType.ACTION, step)
        trace.log(StepType.OBSERVATION, result)
        results.append(result)
    return results, trace

if __name__ == "__main__":
    # Simulate a ReAct run that needs exactly 2 real tool calls before it has enough information
    call_count = {"n": 0}

    def fake_reason(context: str) -> str:
        call_count["n"] += 1
        if call_count["n"] < 3:
            return f"I need more information (step {call_count['n']})"
        return "I now have enough information to answer: the answer is 42"

```

```

def fake_act(thought: str) -> str:
    return f"lookup_tool(query='step {call_count['n']}')"

def fake_is_sufficient(thought: str) -> bool:
    return "enough information to answer" in thought

answer, trace = react_loop("What is the answer?", fake_reason, fake_act, fake_is_sufficient,
max_steps=5)
print(f"Final answer: {answer}")
print(f"Trace length: {len(trace.steps)} steps")
for step in trace.steps:
    print(f" [{step.step_type.value}] {step.content}")

assert "42" in answer
assert len(trace.steps) == 8 # 2 full (thought,action,observation) cycles = 6, + final
(thought,final_answer) pair = 8
print("\nReAct loop verified: converged in 2 tool-call cycles, trace fully inspectable.")

# Verify the termination guard: a reasoner that never becomes sufficient still stops at
max_steps
def never_sufficient(context: str) -> str:
    return "still thinking..."

_, exhausted_trace = react_loop("Unanswerable", never_sufficient, fake_act, lambda t: False,
max_steps=3)
action_steps = [s for s in exhausted_trace.steps if s.step_type == StepType.ACTION]
assert len(action_steps) == 3, "Loop must stop at max_steps, not loop forever"
print(f"Termination guard verified: stopped after {len(action_steps)} action steps, not
indefinitely")

```

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Handling tasks whose correct sequence of steps genuinely depends on information only available at runtime, which a fixed, developer-authored pipeline structurally cannot adapt to.
- **Why Introduced over Legacy Approaches:** A fixed pipeline or a single CoT-reasoning call can only ever reorganize information already available up front; an agentic loop can go fetch genuinely new information mid-task and incorporate it into its next decision, which neither a fixed pipeline nor single-call CoT can do.
- **Key Failure Modes & Limitations:** Unbounded reasoning cycles without an explicit stopping criterion (the `max_steps` guard in the reference code exists specifically to prevent this); non-determinism making the same query potentially take a different path on different runs; and reasoning-quality-dependent action choices — a flawed Thought step directly produces a flawed Action.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Cost scales with the number of Thought/Action/Observation cycles actually taken, bounded by an explicit step limit — worst case is `max_steps` × one cycle's cost, not unbounded, provided a termination guard is actually implemented (the same discipline `03_advanced_rag` Module 08's self-correction loop required).
- **Space/Memory Footprint:** The full trace (all Thoughts/Actions/Observations) typically needs to stay in context for the next step's reasoning, so context size grows with trajectory length — a real, compounding cost distinct

from a fixed pipeline's constant per-call context size.

- **Primary Bottleneck Type:** Latency-bound, specifically on the *variable* number of sequential LLM calls per task — the defining operational difference from a fixed pipeline's constant, predictable latency.
- **Variable Legend:** `max_steps` = the termination guard's bound; no additional closed-form formula variables, per this module's prose/procedural scope.

3. Production & Scalability

- **Deployment Considerations:** Always implement an explicit step/latency/cost bound before deploying any agentic loop — an unbounded loop is a real production incident waiting to happen on a genuinely unanswerable or malformed query (Module 09 covers the production-hardening version of this guard in depth); log the full Thought/Action/Observation trace for every run, since it's the primary tool for debugging a bad agent decision after the fact.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: Why not just always use an agent, since it's strictly more flexible than a fixed pipeline?

Flexibility isn't free — an agent's cost, latency, and reliability are all worse than a fixed pipeline's for a task the pipeline could already handle; the decision framework's whole point is climbing to more flexible (and more expensive/unpredictable) architectures only as far as the task's genuine requirements force you to.

Question: How would you decide between a purely reactive agent and a plan-and-execute agent for a given task?

If the task's structure is genuinely knowable upfront (even if the specific content varies), plan-and-execute reduces wandering and keeps long-horizon tasks on track; if the right next step genuinely can't be known until you see the previous step's result, a purely reactive loop adapts more naturally than a plan committed to too early.

Module 02: Tool Calling & Function Calling Internals

1. Introduction & Intuition

The Core Bottleneck

An LLM's raw output is text. For an agent to actually *do* anything in the world — look something up, write a file, call an API, send a message — that text has to be turned into a structured, executable action, and the result of that action has to be turned back into something the model can reason about. Function calling (tool calling) is the mechanism that makes this round-trip possible: the model is given a set of tool definitions, decides which one (if any) to call and with what arguments, an external system actually executes the call, and the result is fed back into the model's context. Every part of that round-trip is a real engineering surface with its own failure modes — a badly-designed tool schema degrades which tool gets picked, unsafe execution order can corrupt state, and a tool with real side effects that gets called twice by accident can cause real, unrecoverable damage.

High-Level Intuition

Think of tool schemas as a restaurant menu the model has to order from without ever having seen the kitchen. A menu with clear, distinct item names and precise descriptions makes it easy to order the right dish; a menu with ten dishes that all sound similar, vaguely worded, with unclear which toppings are required vs. optional, makes it easy to order the wrong thing even for a customer who knew exactly what they wanted. And once an order is placed, some dishes can be prepared in parallel (a salad and a soup don't depend on each other) while others genuinely can't (you can't plate the dish before it's cooked) — the kitchen's job is knowing which is which, not blindly parallelizing everything or blindly serializing everything.

2. Core Concepts & Mathematical Formulation

Tool Schema Design & Its Effect on Selection Accuracy

Intuition & Practical Use

A tool schema is the model's *only* information about what a tool does and how to call it — it has no other way to know. Four schema-design factors directly, measurably affect how often the model picks the right tool and calls it correctly:

- **Tool naming.** A name like `search` next to another tool named `search_v2` gives the model almost nothing to distinguish them by; `search_internal_wiki` vs. `search_public_web` is unambiguous.
- **Description quality.** A one-line description that doesn't state *when* to use a tool (vs. just *what* it does) leaves the model guessing at applicability from the name alone.
- **Required vs. optional parameter design.** Marking a parameter required when it's genuinely optional forces the model to either fabricate a value or fail the call; marking a genuinely-required parameter optional lets the model skip it and silently produce a malformed or underspecified call.

- **Total number of available tools.** Beyond a moderate count, especially with several similarly-described tools, selection accuracy measurably degrades — the model has to discriminate among a larger, noisier set of options every single call, not just when the "right" tool happens to be among the ambiguous ones.

None of this is fixed by prompting harder after the fact — it's fixed by treating schema design itself as the primary lever for selection accuracy, the same way a well-designed API is easier for a human developer to use correctly than a poorly-documented one.

Safe Parallel vs. Sequential Tool Execution

Intuition & Practical Use

Two tool calls are safe to run in parallel exactly when neither one's input depends on the other's output, and neither has a side effect the other's execution order matters for. Two independent read-only lookups (checking today's weather and checking a stock price) are safe to parallelize — nothing about one call changes what the other should do. A tool call whose argument is the *result* of a previous call must run sequentially, by definition — there's no value to pass it yet. And two tool calls with real side effects that touch the same resource (e.g., two calls that both modify the same record) need sequential execution even if neither's *input* technically depends on the other, specifically to avoid a race condition in what state the resource ends up in. The general principle — build a real dependency graph between planned actions, and only parallelize the parts of that graph with no edge between them — is the same principle Module 06 applies one level up, to whether independent *agents* (not just tool calls) can run concurrently.

Idempotent Tool Execution for Side-Effecting Tools

Intuition & Practical Use

A read-only tool call (checking a price, searching a document) can be safely retried on failure — calling it twice does no harm, since it doesn't change anything. A tool call with a real side effect (charging a payment, sending an email, deleting a record) is a different problem entirely: if the call actually succeeded but the response was lost (a network timeout after the action completed), a naive retry executes the action *again* — a duplicate charge, a duplicate email. Three real mechanisms address this: an **idempotency key** — a unique identifier generated once per logical action and passed with every retry attempt, so the receiving system can recognize "I've already done this" and return the original result instead of repeating the action; a **confirmation gate** — requiring explicit approval (human or a separate policy check) before an irreversible action executes at all, catching a wrong decision *before* it becomes a real side effect rather than after; and **safe-retry design** — only auto-retrying calls that are provably safe to repeat (read-only, or genuinely idempotent by the key mechanism above), never blindly retrying an arbitrary failed side-effecting call.

Hand Calculation: Sequential vs. Parallel Tool-Call Latency

Three independent, safe-to-parallelize tools with real, different execution latencies — 200ms, 400ms, and 600ms — plus a fixed 300ms LLM round-trip overhead per call the model makes.

- **Step 1: Sequential total.** The model must make one LLM call to decide on *each* tool in turn (3 decision calls), plus one final synthesis call once all three results are in — 4 LLM round trips total — plus each tool's own execution time, since nothing overlaps:

$$T_{\text{sequential}} = (n + 1) \times t_{\text{LLM}} + \sum_{i=1}^n t_{\text{tool},i} = (3 + 1) \times 300\text{ms} + (200 + 400 + 600)\text{ms} = 1,200 + 1,200$$

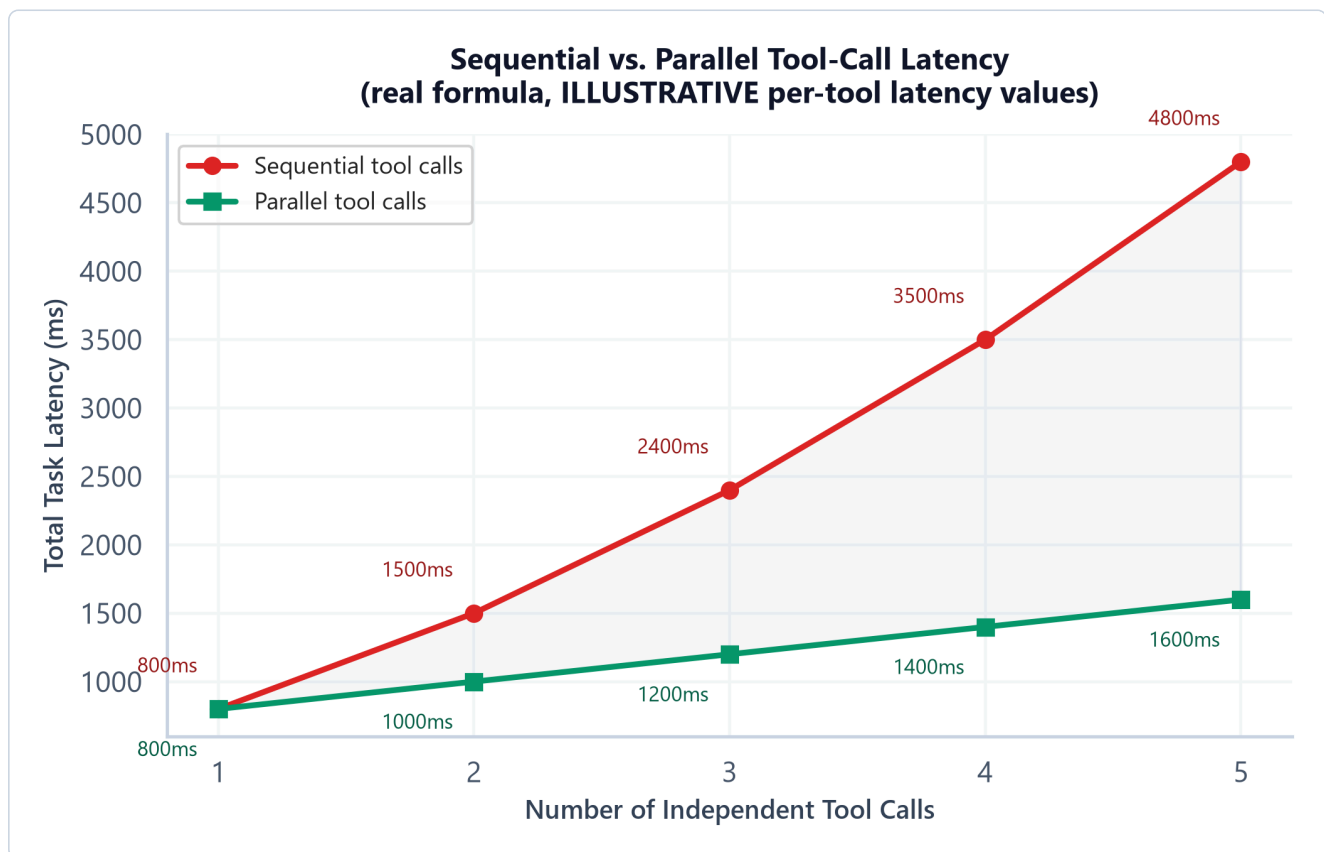
- **Step 2: Parallel total.** The model makes exactly one decision call that requests all three tool calls at once (modern function-calling APIs support this directly), the three tools execute concurrently — the slowest one determines when they're all done — then one final synthesis call:

$$T_{\text{parallel}} = 2 \times t_{\text{LLM}} + \max_i(t_{\text{tool},i}) = 2 \times 300\text{ms} + 600\text{ms} = 600 + 600 = 1,200\text{ms}$$

- **Step 3: Speedup.**

$$\frac{T_{\text{sequential}}}{T_{\text{parallel}}} = \frac{2,400}{1,200} = 2.0\text{x}$$

Two compounding effects produce the 2x speedup here: fewer LLM round trips (2 instead of 4, since the model requests all three tools in one decision call) *and* the tool executions overlapping (600ms, the slowest one, instead of their 1,200ms sum). This is why "safe to parallelize" (the previous section) is the real gate — this speedup only exists because the three tools are genuinely independent; a data dependency between them would force the sequential path regardless of what the round-trip savings might otherwise offer.



- **Plot Interpretation:** The gap between the two lines widens as more independent tools are added — sequential latency grows with both the tool count (more decision round trips) *and* the sum of tool latencies, while parallel latency grows only with the *slowest* individual tool, making the value of safe parallelization increasingly visible as tool count grows. The underlying formula is real; the specific per-tool latency values (200/400/600/800/1000ms) are illustrative example numbers, not measurements.

Hand Calculation: Per-Task Cost Model

A small multi-turn tool-calling task: one initial decision turn requesting the 3 parallel tool calls above (800 input tokens, 150 output tokens), one final synthesis turn incorporating the tool results (1,200 input tokens, 200 output

tokens), generation pricing of \$2.50 per million tokens, and a flat \$0.001 per-call cost for each of the 3 tool APIs.

$$\text{Cost}_{\text{task}} = \sum_{i=1}^{n_{\text{turns}}} (\text{tokens}_{\text{in},i} + \text{tokens}_{\text{out},i}) \times \text{price}_{\text{token}} + \sum_{j=1}^{n_{\text{tools}}} \text{cost}_{\text{tool},j}$$

- **Step 1: Turn 1 (decision) cost.**

$$(800 + 150) \times \$0.0000025 = 950 \times \$0.0000025 = \$0.002375$$

- **Step 2: Turn 2 (synthesis) cost.**

$$(1,200 + 200) \times \$0.0000025 = 1,400 \times \$0.0000025 = \$0.0035$$

- **Step 3: Tool costs.**

$$3 \times \$0.001 = \$0.003$$

- **Step 4: Total task cost.**

$$\$0.002375 + \$0.0035 + \$0.003 = \$0.008875 \approx \$0.0089$$

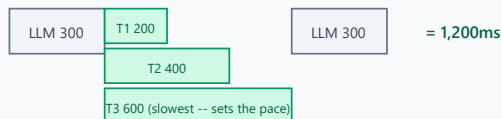
This single number — well under a cent for this task — is what Module 08's "cost per successful task" metric measures in aggregate across many real runs, and what Module 09's production cost budgets are set against; a per-task cost model like this one is the concrete building block both depend on.

Sequential vs. Parallel Tool-Call Timelines (matches the hand calc)

Sequential: 4 LLM round trips, tools run one after another -- 2,400ms total



Parallel: 2 LLM round trips, tools run concurrently -- 1,200ms total



Same 3 tools, same total tool work -- concurrency and fewer round trips are what change, not the work itself.

3. Implementation & Reference Code

Below is a self-contained implementation of the latency/cost hand calculations above, plus a dependency-based safe-parallelization check and an idempotency-key wrapper for side-effecting tool calls.

```
from dataclasses import dataclass, field

@dataclass
class ToolCall:
    name: str
    latency_ms: float
    depends_on: list[str] = field(default_factory=list) # names of tool calls this one needs
    results from
```

```

has_side_effect: bool = False

def sequential_latency_ms(tool_calls: list[ToolCall], llm_overhead_ms: float) -> float:
    """T_sequential = (n+1) * llm_overhead + sum(tool_latencies). Matches the hand calc."""
    n = len(tool_calls)
    return (n + 1) * llm_overhead_ms + sum(t.latency_ms for t in tool_calls)

def parallel_latency_ms(tool_calls: list[ToolCall], llm_overhead_ms: float) -> float:
    """T_parallel = 2 * llm_overhead + max(tool_latencies). Only valid if every call is
    actually safe to parallelize -- callers must check is_safe_to_parallelize first."""
    return 2 * llm_overhead_ms + max(t.latency_ms for t in tool_calls)

def is_safe_to_parallelize(tool_calls: list[ToolCall]) -> bool:
    """Real dependency check: safe only if no call depends on another's output, and no
    two calls with side effects touch (by name convention here) the same resource."""
    names = {t.name for t in tool_calls}
    for t in tool_calls:
        if any(dep in names for dep in t.depends_on):
            return False # a real data dependency exists within this batch
    side_effecting = [t.name for t in tool_calls if t.has_side_effect]
    return len(side_effecting) == len(set(side_effecting)) # no duplicate-resource side effects

@dataclass
class TaskCostModel:
    """Per-task cost model matching the hand calc: sum of (turn tokens * price) + sum of tool
    costs."""
    price_per_token: float
    turns: list[tuple[int, int]] = field(default_factory=list) # (tokens_in, tokens_out) per turn
    tool_costs: list[float] = field(default_factory=list)

    def total_cost(self) -> float:
        llm_cost = sum((tin + tout) * self.price_per_token for tin, tout in self.turns)
        tool_cost = sum(self.tool_costs)
        return llm_cost + tool_cost

class IdempotencyStore:
    """Minimal idempotency-key store: a side-effecting call is only actually executed
    once per key; a retry with the same key returns the original result."""
    def __init__(self):
        self._results: dict[str, str] = {}

    def execute(self, idempotency_key: str, side_effecting_fn) -> str:
        if idempotency_key in self._results:
            return self._results[idempotency_key] # already executed -- return original result,
            don't repeat it
        result = side_effecting_fn()
        self._results[idempotency_key] = result
        return result

if __name__ == "__main__":
    # Hand calc verification: 3 independent tools, 200/400/600ms, 300ms LLM overhead
    tools = [ToolCall("tool1", 200), ToolCall("tool2", 400), ToolCall("tool3", 600)]
    assert is_safe_to_parallelize(tools)

    seq = sequential_latency_ms(tools, llm_overhead_ms=300)

```

```

par = parallel_latency_ms(tools, llm_overhead_ms=300)
print(f"Sequential: {seq:.0f}ms, Parallel: {par:.0f}ms, Speedup: {seq / par:.1f}x")
assert seq == 2400.0
assert par == 1200.0
assert abs(seq / par - 2.0) < 1e-9

# A dependent tool call forces sequential execution regardless of latency savings
dependent_tools = [ToolCall("lookup", 200), ToolCall("act_on_result", 300, depends_on=
["lookup"])]
assert not is_safe_to_parallelize(dependent_tools)
print("Dependency check verified: a real data dependency correctly blocks parallelization")

# Cost hand calc verification
cost_model = TaskCostModel(
    price_per_token=2.50 / 1_000_000,
    turns=[(800, 150), (1200, 200)],
    tool_costs=[0.001, 0.001, 0.001],
)
total = cost_model.total_cost()
print(f"\nTotal task cost: ${total:.6f}")
assert abs(total - 0.008875) < 1e-9

# Idempotency verification: a "retried" side-effecting call executes only once
store = IdempotencyStore()
call_count = {"n": 0}

def charge_payment():
    call_count["n"] += 1
    return f"charged (execution #{call_count['n']})"

first = store.execute("payment-key-abc123", charge_payment)
retried = store.execute("payment-key-abc123", charge_payment) # simulates a retry after a
lost response
print(f"\nFirst call result: {first}")
print(f"Retried call result: {retried}")
assert first == retried == "charged (execution #1)"
assert call_count["n"] == 1, "The side-effecting function must execute exactly once, not once
per retry"
print("Idempotency verified: retry returned the original result without duplicating the side
effect")

```

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Turning a model's text output into a structured, executable action, and getting that action's real-world result back into the model's reasoning — the mechanism that lets an agent do anything beyond generating text.
- **Why Introduced over Legacy Approaches:** Free-text instructions parsed with regex/heuristics are brittle and don't scale past a handful of simple, rigidly-formatted actions; structured function-calling gives the model a machine-checkable schema to conform to, and lets tool-selection accuracy be measured and improved as its own concern.
- **Key Failure Modes & Limitations:** Poor schema design (ambiguous names/descriptions, wrong required/optional design, too many similar tools) directly degrades tool-selection accuracy; unsafe parallelization

of dependent calls corrupts execution order; naive retries of side-effecting calls without idempotency protection cause real, sometimes irreversible duplicate actions.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Sequential tool-call latency is $O(n)$ in both LLM round trips and summed tool latency; parallel execution reduces the tool-latency term to $O(1)$ relative to n (bounded by the slowest single call) when the dependency graph allows it.
- **Space/Memory Footprint:** Every tool call's result typically needs to stay in context for subsequent reasoning, so context size grows with the number of tool round trips in a task — directly connecting to Module 04's context-budget concerns.
- **Primary Bottleneck Type:** Latency-bound on the LLM round-trip count and the critical-path tool latency (the slowest parallel branch, or the full sequential sum); cost-bound on cumulative token usage across turns plus per-call tool API costs.
- **Variable Legend:** n = number of tool calls in a batch, t_{LLM} = LLM round-trip overhead, $t_{\text{tool},i}$ = individual tool execution latency, $\text{price}_{\text{token}}$ = generation token price.

3. Production & Scalability

- **Deployment Considerations:** Treat tool schema design as a first-class, testable artifact — measure tool-selection accuracy on a held-out set of real queries whenever a schema changes, the same discipline as testing any other API contract; require idempotency keys on every side-effecting tool by default, not as an opt-in added after an incident.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: How would you decide whether two tool calls are safe to parallelize?

Build the real dependency graph — does either call's input come from the other's output, and do any two side-effecting calls touch the same resource — and only parallelize the parts of the graph with no such edge; when in doubt, sequential is the safe default.

Question: A payment tool call times out — the request may or may not have actually gone through. How do you handle the retry safely?

Never blindly retry a side-effecting call on ambiguous failure; use an idempotency key generated once for that logical action, so the receiving system can recognize a retry and return the original result instead of executing the charge a second time.

Module 03: Model Context Protocol (MCP) & Agent-Tool Standardization

1. Introduction & Intuition

The Core Bottleneck

Before a standard existed, every application that wanted to connect an LLM to an external tool or data source wrote its own bespoke integration glue — a different shape of code for every combination of application and tool, an $M \times N$ integration problem (M applications, N tools, each pair needing its own glue code). The Model Context Protocol (MCP) exists to collapse that to an $M + N$ problem: any MCP-compliant application can talk to any MCP-compliant server, because both sides speak the same protocol, the same way HTTP lets any browser talk to any web server without custom per-site integration code. That standardization is genuinely valuable — and it also means an MCP server is a new, real integration surface that has to be discovered, trusted, and secured correctly, not just wired up and forgotten.

High-Level Intuition

Before a universal plug standard, every appliance needed its own specific outlet shape, and every building needed to stock every possible outlet type to support every possible appliance. A universal standard means any compliant appliance works with any compliant outlet — but it also means anyone can plug *anything* compliant into that outlet, which is exactly why the outlet needs its own safety mechanisms (breakers, grounding) independent of what gets plugged into it. MCP is the plug standard for agent-tool integration; its security model is the electrical safety layer that has to exist *because* the standard makes connecting things easy.

2. Core Concepts & Mathematical Formulation

This module is architectural/protocol-design throughout — no closed-form calculation applies, consistent with how `03_advanced_rag` treated its own protocol/architecture-survey modules.

MCP Architecture: Client-Host-Server Model

Intuition & Practical Use

MCP defines three roles. The **host** is the application the user actually interacts with (an IDE, a chat client). The **client** lives inside the host and manages a 1:1 connection to a specific server — a host can run multiple clients, one per server it's connected to. The **server** is the external process that actually exposes capabilities — tools to call, resources to read, prompt templates to use — over the protocol. This separation matters because it cleanly decouples *what the host application does with the model* from *what capabilities are available to draw on*, the same separation a web browser (host) and a web server (capability provider) have, mediated by a well-defined protocol rather than custom code per site.

MCP's Three Primitives: Tools, Resources, Prompts

Intuition & Practical Use

A server exposes capabilities through three distinct primitive types, each with a different consumption model: **Tools** are callable functions the model can invoke to take an action or fetch dynamic information — the same function-calling mechanics from Module 02, just standardized in transport. **Resources** are addressable, readable data — files, database records, API responses — that the host can read and provide as context, without necessarily involving a model-initiated tool call at all. **Prompts** are reusable, parameterized prompt templates a server can offer, so common interaction patterns don't need to be re-authored per host application. Keeping these three conceptually distinct matters because they have different trust and update implications: a tool executes code with real side effects, a resource is read-only data, and a prompt is just text — treating a resource as if it could execute like a tool (or vice versa) is a real category error.

Local vs. Remote MCP Servers

Intuition & Practical Use

A **local** MCP server runs as a subprocess on the same machine as the host, typically communicating over stdio — it inherits the host's own OS-level privileges and network access, and there's no third party in the trust chain beyond whatever code the local server itself runs. A **remote** MCP server runs on infrastructure the host doesn't control, typically communicating over SSE/HTTP — connecting to one means trusting a third party's infrastructure, its uptime, and its own security practices, in addition to trusting what the server's tools actually do. The two aren't just a transport detail: a local server's blast radius is bounded by what the local machine can already do; a remote server adds an entirely new trust boundary and a new class of failure (network dependency, third-party compromise) that a local server structurally doesn't have.

Capability Discovery

Intuition & Practical Use

When a client connects to a server, it doesn't come in already knowing what that server offers — it asks. Capability discovery is the handshake where the client queries the server for its available tools, resources, and prompts (with their schemas), *at connection time*, rather than the host application having those hardcoded in advance. This is what makes MCP genuinely dynamic rather than just a fixed API contract: the same host can connect to a completely different server and correctly discover a completely different capability set, with no code change on the host side.

Versioning

Intuition & Practical Use

MCP itself evolves as a protocol, and clients/servers need a way to agree on which version's semantics they're both speaking — a version-negotiation step at connection time, so a newer client can still work with an older server (or fail gracefully and explicitly, rather than silently misinterpreting a newer feature an older server doesn't support). This is the same protocol-evolution problem every widely-adopted standard eventually faces, and getting it wrong means either breaking every existing server on every client update, or every server having to support every historical version forever.

Authentication, Authorization & Tool-Level Permissions

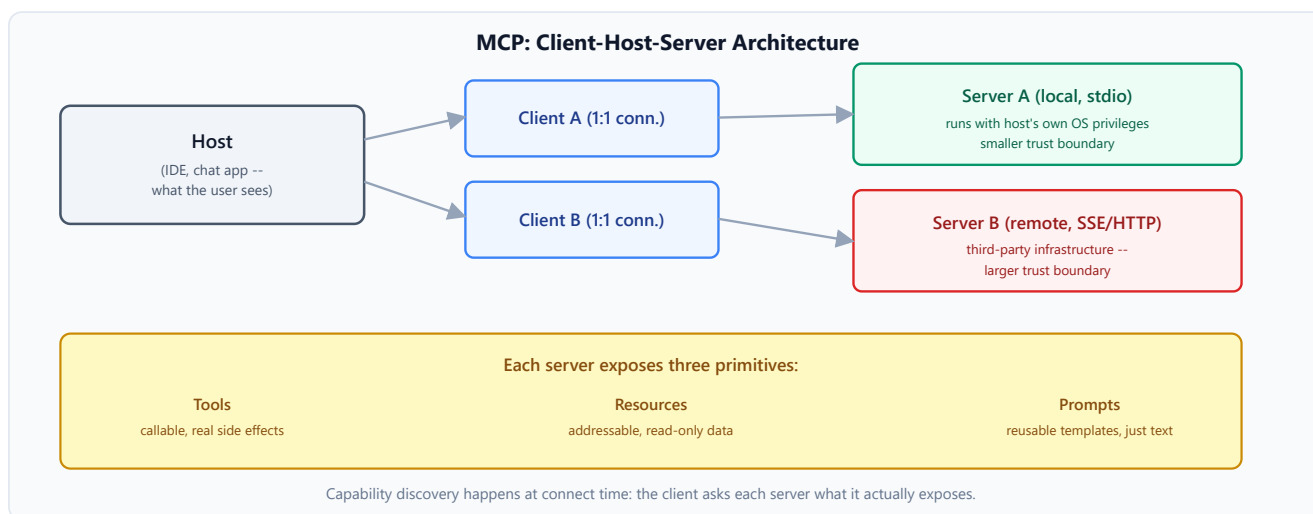
Intuition & Practical Use

Authentication answers "is this client who it claims to be" — a real requirement the moment a server is remote and shared across multiple untrusted or semi-trusted clients. Authorization answers a narrower question: "given who this client is, what is it actually allowed to do" — and critically, this can be scoped *below* the whole-server level, down to individual tools: a client might be authenticated to a server but only authorized to call its read-only tools, not its write/delete tools. Treating "connected to the server" and "allowed to call any of its tools" as the same thing is a real, common mistake — tool-level permission scoping is what prevents a client that only needed read access from ever being *able* to call a destructive tool in the first place, independent of whether it would have chosen to.

Security Risks of Exposing Powerful Tools

Intuition & Practical Use

An MCP server is a new, real attack surface, not just a convenience layer — and the risk scales directly with how powerful the tools it exposes are. A server exposing a read-only "look up today's date" tool has a small blast radius if compromised or misused. A server exposing "execute arbitrary shell commands" or "delete any file" has an enormous one — and because the *model*, not a human, is the one deciding when to call these tools, a server exposing powerful capabilities is only as safe as every layer that constrains when and how the model actually gets to invoke them (this is exactly where Module 09's least-privilege access, authorization boundaries, and sandboxing apply concretely to MCP servers specifically, not just tool-calling in the abstract).



3. Implementation & Reference Code

Below is a minimal, illustrative model of MCP's capability-discovery handshake and tool-level authorization scoping — not a real MCP client/server implementation (the real protocol's JSON-RPC message format is out of scope here), but the *decision logic* both mechanisms depend on.

```
from dataclasses import dataclass, field
from enum import Enum

class PrimitiveType(Enum):
```

```

    TOOL = "tool"
    RESOURCE = "resource"
    PROMPT = "prompt"

@dataclass
class Capability:
    name: str
    primitive_type: PrimitiveType
    required_permission: str # e.g. "read", "write", "admin"

@dataclass
class MCPServer:
    """Illustrative server: exposes a fixed capability set, discoverable at connect time."""
    name: str
    protocol_version: str
    capabilities: list[Capability] = field(default_factory=list)

    def discover_capabilities(self) -> list[Capability]:
        """Capability discovery: the client asks, the server answers -- no hardcoded
        client-side assumption about what this specific server exposes."""
        return list(self.capabilities)

@dataclass
class MCPClient:
    """Illustrative client: connects, discovers capabilities, and enforces the
    connecting principal's authorized permission scope on every call attempt."""
    authorized_permissions: set[str]

    def connect(self, server: MCPServer, client_protocol_version: str) -> list[Capability]:
        if server.protocol_version != client_protocol_version:
            raise ValueError(
                f"Version mismatch: client speaks {client_protocol_version}, "
                f"server speaks {server.protocol_version} -- negotiate or fail explicitly, never
guess."
            )
        return server.discover_capabilities()

    def can_invoke(self, capability: Capability) -> bool:
        """Tool-level authorization: being connected to the server is NOT the same as
        being authorized to call every capability it exposes."""
        return capability.required_permission in self.authorized_permissions

if __name__ == "__main__":
    server = MCPServer(
        name="internal-docs-server",
        protocol_version="2025-06-18",
        capabilities=[
            Capability("search_docs", PrimitiveType.TOOL, required_permission="read"),
            Capability("delete_doc", PrimitiveType.TOOL, required_permission="admin"),
            Capability("doc://readme", PrimitiveType.RESOURCE, required_permission="read"),
        ],
    )

    # A read-only client: authorized for "read" but NOT "admin"
    read_only_client = MCPClient(authorized_permissions={"read"})
    discovered = read_only_client.connect(server, client_protocol_version="2025-06-18")
    print(f"Discovered {len(discovered)} capabilities: {[c.name for c in discovered]}")

```

```

assert len(discovered) == 3

search_tool = next(c for c in discovered if c.name == "search_docs")
delete_tool = next(c for c in discovered if c.name == "delete_doc")
print(f"Can invoke search_docs: {read_only_client.can_invoke(search_tool)}")
print(f"Can invoke delete_doc: {read_only_client.can_invoke(delete_tool)}")
assert read_only_client.can_invoke(search_tool) is True
assert read_only_client.can_invoke(delete_tool) is False
print("Tool-level authorization verified: connection alone did not grant delete access.")

# A version mismatch fails explicitly, rather than silently misinterpreting the server
try:
    read_only_client.connect(server, client_protocol_version="2024-01-01")
    raise AssertionError("Should have raised on version mismatch")
except ValueError as e:
    print(f"\nVersion mismatch correctly rejected: {e}")

```

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Collapsing the $M \times N$ bespoke-integration problem (M applications, N tools) into an $M + N$ standardized-protocol problem, so any compliant host can use any compliant server without custom per-pair glue code.
- **Why Introduced over Legacy Approaches:** Framework-specific tool abstractions (e.g., LangChain tools) work well within that one framework but don't transfer to a different host application without re-implementation; native LLM function-calling standardizes the model-facing schema but says nothing about how a whole ecosystem of tool *providers* should be discovered, versioned, or authorized across many different host applications.
- **Key Failure Modes & Limitations:** A poorly-secured server exposing powerful tools is a real, direct attack surface; a version mismatch handled by silent guessing instead of explicit negotiation risks misinterpreting a newer server's capabilities; treating server-level connection as equivalent to tool-level authorization opens a real permission-scoping gap.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Not compute-bound — MCP's overhead is protocol/network round-trip cost (connection handshake, capability discovery, per-call request/response), not model FLOPs.
- **Space/Memory Footprint:** Minimal — a client maintains its discovered capability list and connection state per server; the real footprint concern is on the server side, scoped to whatever it exposes.
- **Primary Bottleneck Type:** Latency-bound on the protocol round trip for remote (SSE/HTTP) servers specifically; a local (stdio) server avoids network latency entirely but inherits the host's own OS-level privileges, which is a security trade-off, not a performance one.
- **Variable Legend:** No closed-form formula variables, per this module's prose/protocol-design scope.

3. Production & Scalability

- **Deployment Considerations:** Scope tool-level permissions to the minimum a given client actually needs, independent of what the whole server exposes; treat a remote MCP server as a genuinely new trust boundary

requiring its own security review, not an extension of the host's existing trust; handle protocol version mismatches by explicit negotiation/failure, never silent best-effort guessing.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: Why is a remote MCP server a bigger security concern than a local one, even if they expose the identical tool set?

A local server's blast radius is bounded by what the local machine can already do and runs under the host's own privileges with no third party involved; a remote server adds a third-party infrastructure dependency and a genuinely new trust boundary — its uptime, its own security practices, and the network path to it are all new risk surface a local server doesn't have.

Question: How does MCP relate to native LLM function-calling — does it replace it?

No — MCP standardizes *how tools are discovered and transported* across the whole agent-tool ecosystem; the model still ultimately performs function-calling (Module 02's mechanics) against whatever tool schema the MCP capability discovery handshake surfaced, so the two are complementary, not competing.

Module 04: Context, State & Memory for Agents

1. Introduction & Intuition

The Core Bottleneck

"Context," "state," and "memory" are the single most commonly conflated set of terms in agent system design — used interchangeably in casual conversation, but they answer three genuinely different questions, and conflating them produces real design mistakes: storing something as ephemeral state when it needed to survive across sessions (it's lost the moment the run ends), or storing everything as persistent memory when most of it was only ever needed for the current run (bloating a memory store with noise nobody will ever retrieve). Getting this distinction precise is a prerequisite for the rest of this module, not a pedantic warm-up.

High-Level Intuition

Think of a single work session at a desk. **Context** is everything actually spread out on the desk in front of you right now — the documents you're looking at this exact moment. **State** is your current progress on the task at hand — which step of the process you're on, what you've already tried — the kind of thing you'd need a colleague to know if they had to take over from you mid-task without starting from scratch. **Memory** is what you deliberately write down in your notebook specifically because you know you'll need it again next week, after this session is long over and the desk has been cleared. All three matter, but they're not the same thing, and an agent system that doesn't distinguish them will confuse "what does the model see right now" with "what does the model need to resume this task" with "what should genuinely outlive this task."

2. Core Concepts & Mathematical Formulation

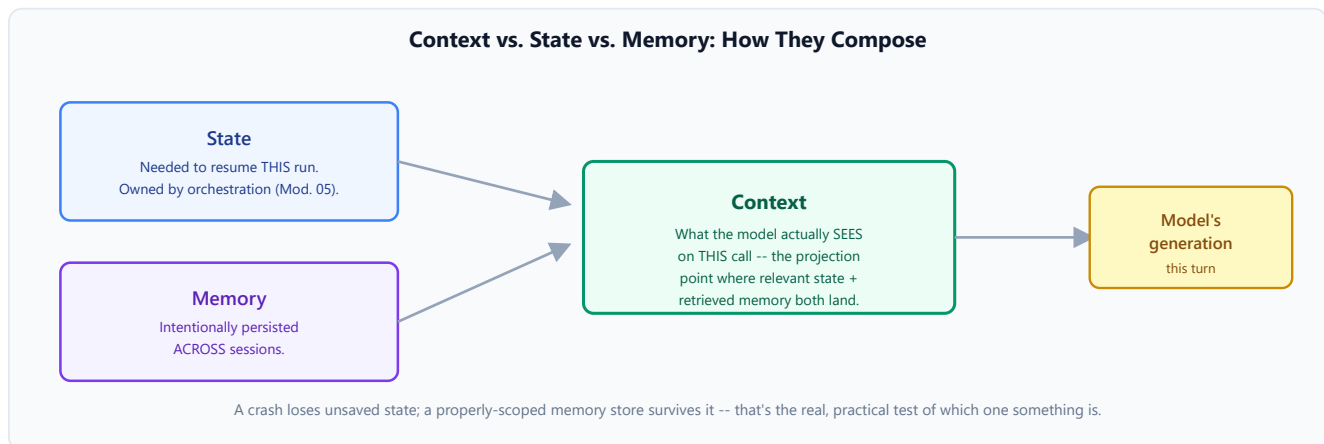
The Three-Way Distinction: Context vs. State vs. Memory

Intuition & Practical Use

- **Context** is the actual information sent to the model on a given call — the assembled prompt: system instructions, tool schemas, retrieved memory, and whatever state is relevant right now. Context is *what the model can literally see this turn*, nothing more.
- **State** is the information required to execute or resume the current workflow — often ephemeral or checkpointed, scoped to one run, and owned by the orchestration layer (Module 05's subject). If the process crashes mid-task, state is what a checkpoint restores so the workflow can resume without starting over.
- **Memory** is information *intentionally* persisted across interactions or sessions — deliberately written and deliberately retrieved, not automatically carried forward the way state is within a single run.

The three compose rather than sit as independent layers: on any given turn, **context is the projection point** where relevant state and retrieved memory both get assembled into what the model actually sees. Get this wrong and two distinct classes of bug appear: something stored only as ephemeral state silently vanishes the moment a run ends, even though it was actually needed next week (it should have been memory); or a memory store gets flooded with

information that was only ever relevant to one specific run's internal progress and nobody will ever usefully retrieve later (it should have stayed state).



Short-Term vs. Long-Term Memory

Intuition & Practical Use

Short-term memory (a conversation buffer, a sliding window of recent turns) lives naturally within a single session and is usually just carried forward turn to turn with no separate storage system — it's really an implementation detail of how a run's context gets built up, sitting close to state in practice even though conceptually it's still memory (it's still deliberately retained information, just retained for the duration of one session). Long-term memory is explicitly persisted to outlive the session entirely — the next time a user starts a new conversation, long-term memory is what lets the agent recall something from weeks ago that a fresh session's empty conversation buffer never would.

Episodic vs. Semantic Memory

Intuition & Practical Use

Episodic memory is memory of specific *events* — "the user asked about X on Tuesday and I told them Y." Semantic memory is memory of general *facts* extracted and generalized from those events — "this user prefers concise answers," a fact that might have been inferred from many episodes rather than stored as any one of them. Episodic memory answers "what happened"; semantic memory answers "what do I now know," having already generalized past the specific episode that taught it. Production memory systems often maintain both: raw episodic logs for traceability, plus a distilled semantic layer that's cheaper to retrieve from and doesn't require re-deriving the same generalization on every query.

Vector-Store-Backed Long-Term Memory: A RAG-Adjacent Problem

Intuition & Practical Use

Retrieving the *relevant* slice of a large long-term memory store for the current context is structurally the same problem `03_advanced_rag` solves for document retrieval — embed memories, index them, retrieve the nearest ones to the current query/context. This module doesn't re-derive that mechanism; it's the same embeddings, vector indexing, and hybrid retrieval machinery, just applied to a memory store instead of a document corpus. What's specific to *agent* memory rather than generic RAG is the write side — deciding what's worth writing to memory in the first place (the next subsection) — which document-retrieval RAG doesn't need to solve, since its corpus is typically given, not accumulated by the system's own behavior.

Memory Summarization & Compression

Intuition & Practical Use

Context windows are finite, and a long-running conversation's full history eventually won't fit. Memory summarization compresses older turns into a shorter summary that preserves the gist while discarding verbatim detail — trading recall completeness (the compressed version can't answer a question about an exact quote from ten turns ago) for staying within budget. *When* to trigger this compression is a real, quantifiable decision, not a vague "when it gets long" judgment call.

Memory Write Policies

Intuition & Practical Use

Not everything that happens during a run is worth remembering — writing indiscriminately turns a memory store into noise that dilutes genuinely useful retrieval later. A write policy is the explicit rule for what gets persisted to long-term memory at all: a user's stated preference (durable, worth remembering), versus an incidental detail of one specific task's internal progress (state, not memory — it shouldn't outlive the run in the first place, tying directly back to this module's opening distinction).

Hand Calculation: When to Trigger Memory Summarization

An agent with an 8,000-token context window, a fixed 500-token system-prompt-plus-tool-schema overhead, a 300-token reserved budget for the next turn's own content, a summarization threshold of 80% of the window, and conversation history growing by roughly 350 tokens per turn (user message plus assistant response, averaged).

- **Step 1: Compute the absolute token threshold.**

$$\theta \times \text{context_window} = 0.8 \times 8,000 = 6,400 \text{ tokens}$$

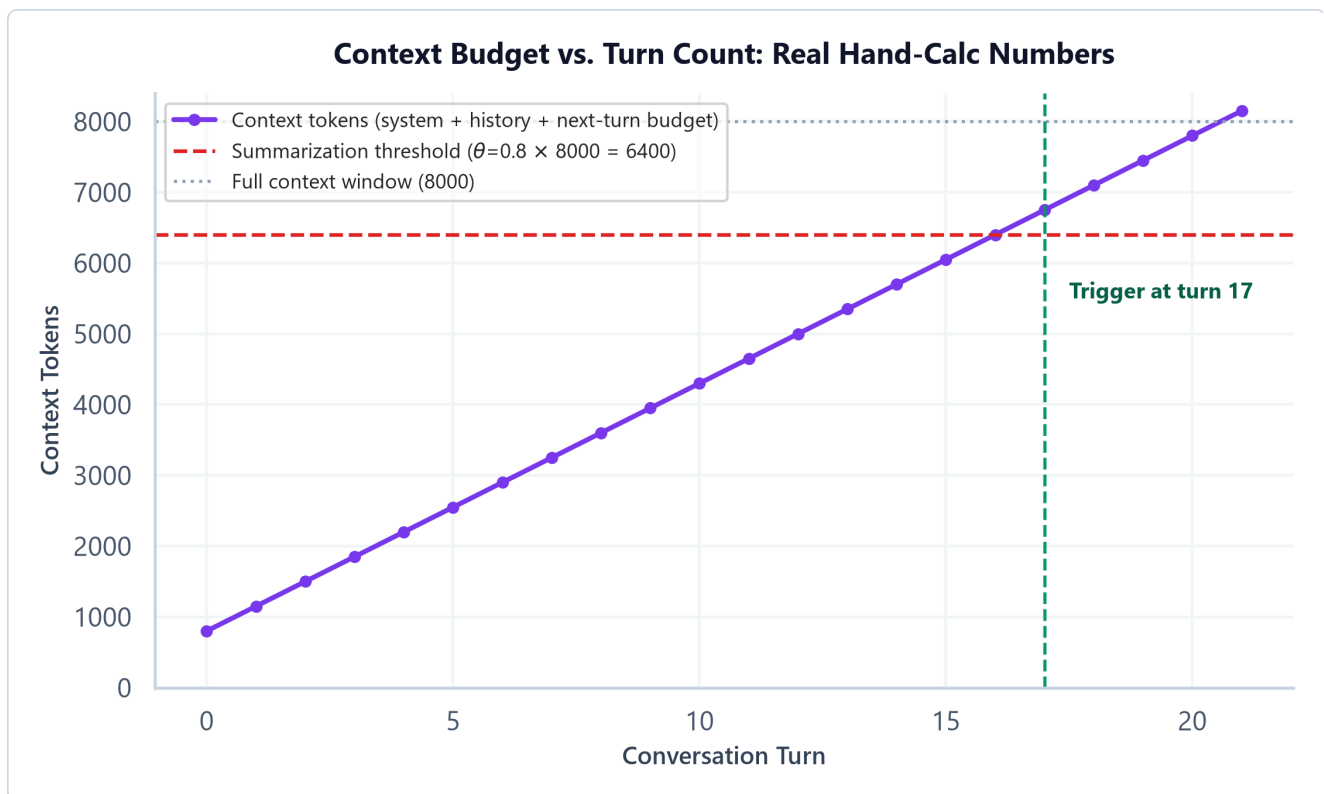
- **Step 2: Subtract the fixed overhead to find the history budget.**

$$\text{history_budget} = 6,400 - (\text{tokens}_{\text{system}} + \text{tokens}_{\text{next_turn_budget}}) = 6,400 - (500 + 300) = 5,600 \text{ tokens}$$

- **Step 3: Solve for the triggering turn.**

$$\text{turn}_{\text{trigger}} = \left\lceil \frac{\text{history_budget}}{\text{tokens_per_turn}} \right\rceil + 1 = \left\lceil \frac{5,600}{350} \right\rceil + 1 = 16 + 1 = 17$$

At exactly turn 16, accumulated history is $350 \times 16 = 5,600$ tokens — precisely *at* the budget, not yet *over* it, so the trigger condition ($>$, strictly greater) doesn't fire yet. Turn 17 pushes history to 5,950 tokens, the first point that genuinely exceeds the threshold — summarization must trigger there to keep total context under the 6,400-token budget with room to spare before hitting the hard 8,000-token ceiling.



- **Plot Interpretation:** The context-token line grows linearly with turn count until the trigger point, where summarization would compress the accumulated history and reset the growth — the plot shows the real, computed pre-summarization trajectory and exactly where it crosses the real threshold, not an illustrative shape.

3. Implementation & Reference Code

Below is a self-contained implementation of the summarization-trigger hand calculation, plus a minimal illustration of the Context/State/Memory distinction as three separate, explicitly-scoped stores.

```
import math
from dataclasses import dataclass, field

def summarization_trigger_turn(context_window: int, theta: float, tokens_system: int,
                              tokens_next_turn_budget: int, tokens_per_turn: int) -> int:
    """Solves for the first turn where accumulated history would push total context
    strictly over theta * context_window. Matches the hand calculation above."""
    threshold = theta * context_window
    history_budget = threshold - (tokens_system + tokens_next_turn_budget)
    assert history_budget > 0, "Fixed overhead alone already exceeds the summarization threshold"
    turns_to_reach_budget = history_budget / tokens_per_turn
    return math.ceil(turns_to_reach_budget) + 1 # first turn that strictly exceeds, not just
    reaches

@dataclass
class RunState:
    """STATE: scoped to one run, needed to resume it, NOT intended to outlive the run."""
    run_id: str
    current_step: int = 0
    scratch_data: dict = field(default_factory=dict)
```

```

@dataclass
class LongTermMemory:
    """MEMORY: intentionally persisted, explicitly written, survives across runs/sessions."""
    entries: list[str] = field(default_factory=list)

    def write(self, fact: str, write_policy) -> bool:
        """Only persist what the write policy actually judges durable -- never write
        indiscriminately, per the Memory Write Policies section above."""
        if write_policy(fact):
            self.entries.append(fact)
            return True
        return False

def assemble_context(system_prompt: str, run_state: RunState, memory: LongTermMemory,
                    current_turn_input: str, memory_query_fn) -> str:
    """CONTEXT: the actual assembled prompt sent to the model this turn -- the
    projection point where relevant state and retrieved memory both land."""
    relevant_memories = memory_query_fn(memory.entries, current_turn_input)
    return (
        f"[SYSTEM] {system_prompt}\n"
        f"[STATE] step={run_state.current_step}, scratch={run_state.scratch_data}\n"
        f"[RETRIEVED MEMORY] {relevant_memories}\n"
        f"[CURRENT TURN] {current_turn_input}"
    )

if __name__ == "__main__":
    # Hand calc verification
    trigger_turn = summarization_trigger_turn(
        context_window=8000, theta=0.8, tokens_system=500,
        tokens_next_turn_budget=300, tokens_per_turn=350,
    )
    print(f"Summarization must trigger at turn: {trigger_turn}")
    assert trigger_turn == 17

    # Context/State/Memory distinction, illustrated as three genuinely separate stores
    state = RunState(run_id="run-42", current_step=2, scratch_data={"partial_result": "draft v1"})
    memory = LongTermMemory()

    def write_policy(fact: str) -> bool:
        # Only durable, cross-session-relevant facts get persisted -- not run-internal scratch
        data
        return "preference" in fact or "policy" in fact

    memory.write("User prefers concise, bulleted answers (preference)", write_policy)
    memory.write("Draft v1 of this specific task's output (scratch, not durable)", write_policy)
    print(f"\nMemory entries written: {len(memory.entries)}")
    assert len(memory.entries) == 1 # only the durable preference was written, not the scratch
    note
    print(f"Kept: {memory.entries[0]}")

    def fake_memory_query(entries, query):
        return [e for e in entries if "preference" in e]

    context = assemble_context(
        system_prompt="You are a helpful assistant.",
        run_state=state,
        memory=memory,

```

```

    current_turn_input="What's the weather like?",
    memory_query_fn=fake_memory_query,
)
print(f"\nAssembled context:\n{context}")
assert "[STATE]" in context and "[RETRIEVED MEMORY]" in context
print("\nContext/State/Memory distinction verified: state and memory are separate stores that
both project into one assembled context.")

```

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Giving an agent both the ability to resume a single run correctly after an interruption (state) and the ability to recall genuinely durable information across entirely separate runs (memory), without conflating the two or blowing the context budget assembling either into what the model sees (context).
- **Why Introduced over Legacy Approaches:** A single undifferentiated "conversation history" treats everything as equally durable and equally scoped, which either loses information that should have survived (treated as ephemeral state when it needed to be memory) or bloats retrieval with information that should never have outlived its run (treated as memory when it was really just state).
- **Key Failure Modes & Limitations:** Storing run-internal scratch data as long-term memory pollutes future retrieval with noise; storing genuinely durable facts only as ephemeral state loses them the moment a run ends; assembling too much state/memory into context blows the token budget (the direct subject of this module's hand calculation).

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Memory retrieval is a real, separate vector-search cost (Module 04's own vector-store-backed retrieval, borrowing `03_advanced_rag`'s ANN mechanics) paid on top of the model's own generation cost every turn that needs memory recall.
- **Space/Memory Footprint:** State is typically small and transient, cleared or archived once a run completes; long-term memory grows monotonically over a system's lifetime unless an explicit retention/pruning policy exists, making unbounded memory growth a real, separate operational concern from context-window budgeting.
- **Primary Bottleneck Type:** Context assembly is token-budget-bound (this module's hand calculation); memory retrieval at scale is the same latency/recall trade-off `03_advanced_rag` Module 04 covers for ANN search, just applied to a memory index instead of a document index.
- **Variable Legend:** θ = summarization-trigger threshold as a fraction of the context window, `tokens_system / tokens_next_turn_budget` = fixed per-turn overhead, `tokens_per_turn` = average history growth rate per turn.

3. Production & Scalability

- **Deployment Considerations:** Decide the Context/State/Memory classification of every piece of information explicitly at design time, not implicitly by whatever happens to be convenient to store where; instrument the real running token count against the summarization threshold rather than triggering compression on a fixed turn-count heuristic that ignores how verbose individual turns actually are.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: A user reports the agent "forgot" something they mentioned last week. How would you debug whether this is a state, memory, or context problem?

First check whether the information was ever actually written to long-term memory at all (a write-policy gap) versus written but not retrieved into this turn's context (a retrieval gap) versus never intended to persist because it was only ever run-scoped state — each is a different root cause requiring a different fix.

Question: Why not just always include the full conversation history in context and skip summarization entirely?

Context windows are finite, and even within budget, "lost in the middle" effects (the same phenomenon `03advancedrag`` Module 01 covers for retrieved context) mean an ever-growing raw history doesn't just cost more tokens, it can genuinely degrade the model's ability to use the most relevant parts of it.

Module 05: Agent Orchestration, State Machines & Durable Execution

1. Introduction & Intuition

The Core Bottleneck

Module 01's ReAct loop is a *procedure*: reason, act, observe, repeat, implicitly, inside one running process. That's fine until the process itself can't be trusted to keep running uninterrupted for the task's full duration — a crash, a deploy, a multi-hour task that outlives any single process's reasonable lifetime, or a step that genuinely needs a human to look at it before continuing. An implicit loop has no answer to any of these; it just dies with the process. Making the control flow an *explicit* graph — real nodes, real edges, real persisted state at each step — is what makes an agent's execution debuggable, interruptible, and durable across exactly the kinds of real-world interruptions a production system has to survive.

High-Level Intuition

An implicit reasoning loop is like doing a multi-step task entirely in your head, from memory, with nothing written down — if you get interrupted, you've lost your place, and there's no way for someone else to pick up where you left off. A graph-based orchestration with checkpointing is like keeping a written, step-by-step log as you go: if you get pulled away mid-task, anyone (including you, later) can read the log, see exactly which step you'd completed, and resume from there instead of starting over. The log itself is the durability mechanism — it's what turns "I was doing something" into "I can prove exactly what I'd done and pick it back up."

2. Core Concepts & Mathematical Formulation

This module stays architectural/procedural throughout — orchestration topology and durable-execution mechanics are structural concepts, not closed-form calculations, consistent with `03_advanced_rag` Module 08's Agentic RAG treatment of its own orchestration-adjacent material.

Graph-Based Orchestration: Nodes, Edges & Shared State

Intuition & Practical Use

Instead of one implicit loop, an agent's control flow is modeled as an explicit graph: nodes are units of work (a reasoning step, a tool call, a sub-agent invocation), edges define what can follow what, and a shared state object flows through the graph, read and updated by whichever node is currently executing. This is strictly more explicit than Module 01's implicit loop — every possible transition is a real, inspectable edge in the graph, not an emergent property of whatever the model happened to decide at runtime — which is exactly what makes a graph's execution *debuggable* in a way an implicit loop structurally isn't: you can look at the graph definition and know every path execution could have taken, before it ever runs.

Conditional Routing & Cyclic Graphs

Intuition & Practical Use

Conditional routing lets an edge's destination depend on the current state — "if the tool call succeeded, go to synthesis; if it failed, go to retry" — rather than every node having exactly one fixed successor. Cyclic graphs (edges that loop back to an earlier node) are what let a graph express genuinely iterative agent behavior — a retry loop, a refine-until-satisfied loop — the graph-based equivalent of Module 01's ReAct cycle, just made explicit as a real loop in the graph's own structure instead of an implicit "call the model again" pattern.

Human-in-the-Loop Interrupts

Intuition & Practical Use

Some actions genuinely warrant a pause for human approval before they execute — the same confirmation-gate principle from Module 02's idempotent-execution coverage, expressed at the orchestration level: a node can explicitly interrupt the graph's execution, surface its proposed next action to a human, and wait — the graph's state sits durably paused until an explicit resume signal arrives, which could be seconds or days later. This only works at all because the graph's state is genuinely persisted while paused, not held only in an ephemeral in-memory loop that dies the moment nothing is actively running.

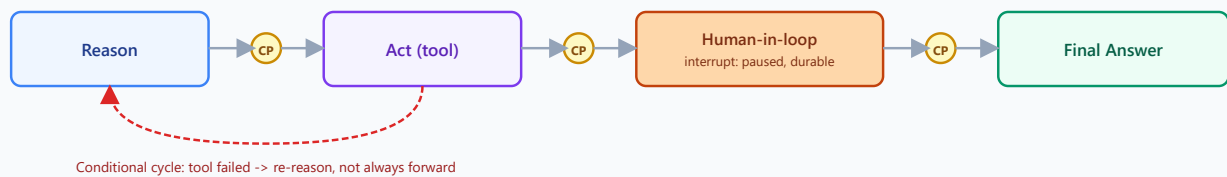
Durable Execution as a First-Class Topic

Intuition & Practical Use

Durable execution is the set of guarantees that let a long-running, graph-based workflow survive real-world interruptions correctly, not just resume by luck:

- **Checkpointing** — persisting the graph's state at each step (or at defined checkpoint boundaries), so there's always a durable, recoverable record of exactly how far execution had gotten.
- **Crash recovery** — what the workflow needs to actually survive an unexpected process or node failure mid-run: the checkpoint has to capture *everything* needed to resume correctly, not just a partial snapshot that looks complete but is missing something the resumed execution silently depends on.
- **Resume** — continuing execution from the last good checkpoint rather than restarting the entire workflow from the beginning; this is the entire practical payoff of checkpointing, and checkpointing without a working resume path is just unused storage.
- **Workflow-level idempotency** — re-running a step that gets resumed after a checkpoint must not duplicate that step's effects if it had actually already completed before the crash; this is the same idempotency principle Module 02 establishes for individual tool calls, now applied at the level of an entire workflow step, which may itself contain several tool calls.
- **Pause/resume** — the general mechanism human-in-the-loop interrupts (above) are a specific instance of: durably suspending execution and later continuing it, on a schedule or an external signal, not just an unplanned crash.
- **Long-running workflows** — tasks that may genuinely span hours or days, where the process that started the workflow may not even stay alive the whole time; durable execution here means the workflow's state lives independently of any one process's lifetime, in a persisted store a *different* process can pick up and continue from.

Graph-Based Orchestration with Durable Checkpoints



CP = checkpoint: state persisted here, independent of any one process's lifetime

If the process crashes between two checkpoints, resume restarts from the LAST checkpoint --
workflow-level idempotency ensures that resumed step doesn't duplicate work already completed before the crash.

Comparison Against Simpler Linear Orchestration

Intuition & Practical Use

A simple linear chain (do A, then B, then C, no branching, no cycles) needs none of this machinery — it's easier to reason about and cheaper to build precisely because it doesn't need conditional routing, cycles, or (usually) durable checkpointing at every step. The added complexity of graph-based orchestration with full durable execution is justified specifically by: genuine conditional branching the task requires, genuine iterative/cyclic behavior, or a task long-running/critical enough that surviving a crash without losing progress actually matters. Applying this level of machinery to a task that was always going to execute the same three steps in the same order, quickly, is real, unjustified overhead — the same "climb the complexity ladder only as far as required" discipline Module 01's decision framework applies one level up.

3. Implementation & Reference Code

Below is a minimal, illustrative graph orchestrator with checkpointing, crash-simulated resume, and workflow-level idempotency — not a production framework (real systems like LangGraph provide this durably and at scale), but the mechanics both depend on.

```
from dataclasses import dataclass, field
from enum import Enum

class NodeStatus(Enum):
    PENDING = "pending"
    COMPLETED = "completed"
    INTERRUPTED = "interrupted" # paused for human-in-the-loop approval

@dataclass
class Checkpoint:
    node_name: str
    state_snapshot: dict
    status: NodeStatus

@dataclass
class DurableGraphRun:
```

```

"""Minimal durable graph executor: persists a checkpoint after every node,
and resume() continues from the last checkpoint rather than restarting."""
checkpoints: list[Checkpoint] = field(default_factory=list)
state: dict = field(default_factory=dict)
completed_side_effects: set[str] = field(default_factory=set) # workflow-level idempotency
guard

def run_node(self, node_name: str, node_fn, requires_approval: bool = False) -> NodeStatus:
    """Executes one node, checkpointing afterward. Skips re-executing a node whose
    side effect (by name) already completed -- the workflow-level idempotency guard."""
    if node_name in self.completed_side_effects:
        return NodeStatus.COMPLETED # already done before a prior crash -- don't repeat it

    if requires_approval:
        self.checkpoints.append(Checkpoint(node_name, dict(self.state),
NodeStatus.INTERRUPTED))
        return NodeStatus.INTERRUPTED # durably paused; caller must explicitly resume

    result = node_fn(self.state)
    self.state.update(result)
    self.completed_side_effects.add(node_name)
    self.checkpoints.append(Checkpoint(node_name, dict(self.state), NodeStatus.COMPLETED))
    return NodeStatus.COMPLETED

def last_checkpoint(self) -> Checkpoint | None:
    return self.checkpoints[-1] if self.checkpoints else None

def simulate_crash_and_resume(run: DurableGraphRun) -> DurableGraphRun:
    """Simulates a crash: a NEW DurableGraphRun object rebuilt purely from the last
    persisted checkpoint's state, proving resume doesn't depend on the original
    in-memory process still being alive."""
    last = run.last_checkpoint()
    assert last is not None, "Cannot resume a run with no checkpoints"
    resumed = DurableGraphRun(
        checkpoints=list(run.checkpoints),
        state=dict(last.state_snapshot),
        completed_side_effects=set(run.completed_side_effects),
    )
    return resumed

if __name__ == "__main__":
    call_log = []

    def reason_node(state: dict) -> dict:
        call_log.append("reason")
        return {"plan": "call search tool"}

    def act_node(state: dict) -> dict:
        call_log.append("act")
        return {"tool_result": "found: 42"}

    def synthesise_node(state: dict) -> dict:
        call_log.append("synthesise")
        return {"final_answer": f"The answer is derived from: {state['tool_result']}"}

    # Run through reason -> act, then simulate a crash before synthesise ever runs
    run = DurableGraphRun()
    run.run_node("reason", reason_node)
    run.run_node("act", act_node)

```



```

print(f"Before crash -- nodes executed: {call_log}")
print(f"Checkpoints persisted: {[c.node_name for c in run.checkpoints]}")
assert call_log == ["reason", "act"]

# Simulate the crash: rebuild a fresh run purely from the last checkpoint
resumed_run = simulate_crash_and_resume(run)
print(f"\nResumed from checkpoint: {resumed_run.last_checkpoint().node_name}")
print(f"Resumed state: {resumed_run.state}")
assert resumed_run.state == {"plan": "call search tool", "tool_result": "found: 42"}

# Resume: re-running "act" must NOT duplicate its side effect (workflow-level idempotency)
status = resumed_run.run_node("act", act_node)
print(f"\nRe-running 'act' after resume: status={status.value}, call_log={call_log}")
assert call_log == ["reason", "act"], "act must NOT have executed again -- idempotency guard held"
assert status == NodeStatus.COMPLETED

# Now genuinely proceed to the next real node
resumed_run.run_node("synthesize", synthesize_node)
print(f"\nFinal state: {resumed_run.state['final_answer']}")
assert call_log == ["reason", "act", "synthesize"]
print("\nDurable execution verified: resumed from checkpoint, no duplicated side effects, workflow completed correctly.")

```

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Making an agent's control flow explicit, inspectable, and durable across real-world interruptions (crashes, deploys, human-approval pauses, tasks that outlive a single process), which an implicit reasoning loop structurally cannot survive.
- **Why Introduced over Legacy Approaches:** An implicit loop (Module 01's ReAct pattern, run inside one continuously-executing process) has no mechanism to persist its progress or resume after an interruption; making state, transitions, and checkpoints explicit is what turns "hope the process doesn't die" into an actual, testable durability guarantee.
- **Key Failure Modes & Limitations:** A checkpoint that doesn't capture everything the resumed execution depends on silently produces incorrect resumed behavior; resuming a step without workflow-level idempotency duplicates that step's side effects; unjustified graph/durability complexity on a task that was always going to be a simple linear sequence adds real engineering overhead for no benefit.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Not compute-bound by the orchestration mechanism itself — the graph adds a small, constant per-node bookkeeping overhead (checkpoint write) on top of whatever the node's actual work (an LLM call, a tool call) costs.
- **Space/Memory Footprint:** Checkpoint storage grows with the number of steps in a workflow and the size of the state snapshotted at each one — a genuinely real, scaling storage cost for long-running or high-volume workflows, not a rounding error.
- **Primary Bottleneck Type:** Storage/latency-bound on the checkpoint write itself (a real I/O cost on every step, not free), traded directly against the durability guarantee it buys — a workflow that checkpoints too infrequently

risks losing more progress on a crash; one that checkpoints on every trivial sub-step pays real, possibly unnecessary I/O cost.

- **Variable Legend:** No closed-form formula variables, per this module's prose/procedural scope.

3. Production & Scalability

- **Deployment Considerations:** Checkpoint at meaningful step boundaries, not so finely that I/O cost dominates or so coarsely that a crash loses substantial progress; ensure every checkpoint captures everything the resumed execution actually depends on (test this explicitly by simulating a resume from each checkpoint, as the reference code does, not just trusting it works); design every resumable node to be safe to re-run (workflow-level idempotency), since "the crash happened exactly between two checkpoints" is not a rare edge case at production scale, it's a routine occurrence.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: How would you test that your durable-execution/resume logic actually works, not just that it doesn't error?

Explicitly simulate a crash at every checkpoint boundary — rebuild a fresh execution purely from each persisted checkpoint's state (never reusing the original process's in-memory objects) and assert the resumed run produces the same final result with no duplicated side effects, exactly the pattern in this module's own reference code.

Question: When is a simple linear chain actually the right choice over a full durable graph?

When the task has no genuine conditional branching or iterative behavior, and isn't long-running or critical enough that surviving a mid-execution crash without progress loss actually matters — the same complexity-ladder discipline as Module 01's agent-vs-pipeline decision framework, applied to orchestration specifically.

Module 06: Multi-Agent Systems & Coordination Patterns

1. Introduction & Intuition

The Core Bottleneck

A single agent, however well-designed, is still one line of reasoning trying to hold an entire task in its head at once. Some tasks genuinely benefit from splitting into specialized sub-problems — a research task and a writing task call for different skills, different tools, even different prompting strategies, and forcing one generalist agent to do both can produce worse results than two agents each focused on one job. But splitting into multiple agents is not automatically an improvement: it introduces coordination overhead, new cross-agent failure modes, and direct cost multiplication that a single well-designed agent simply doesn't have. This module is about the coordination *mechanisms* that make multi-agent systems work when they're actually the right choice — and Module 01's decision framework, not intuition, is what should decide whether they're the right choice at all.

High-Level Intuition

A single skilled generalist can handle a moderately complex project alone. A team of specialists can handle a genuinely complex one *better* — if the team actually coordinates well. But a poorly-coordinated team of specialists, each doing their own piece without clear hand-offs, can easily produce worse results than the generalist working alone — redundant work, contradictory outputs, and time lost to miscommunication that never existed when it was just one person deciding everything themselves. Multi-agent systems inherit exactly this dynamic: the specialization can genuinely help, but only if the coordination mechanism connecting the specialists is actually sound.

2. Core Concepts & Mathematical Formulation

This module stays architectural throughout — coordination topology is a structural/comparative concept, consistent with how this topic treats orchestration patterns generally; the one quantitative artifact in this module (the cost-scaling plot below) is explicitly labeled illustrative, not a formula this module introduces as core.

Orchestrator-Worker (Supervisor) Patterns

Intuition & Practical Use

One coordinating agent (the orchestrator/supervisor) breaks the overall task into sub-tasks, dispatches each to a specialized worker agent, and assembles their results into a final output. The orchestrator owns the overall plan and the hand-off logic; workers own their specific sub-task and don't need to know about each other at all. This is the more controllable of the two major topologies — there's one place (the orchestrator) where the overall task's progress and coordination logic lives, which makes debugging and reasoning about the system's behavior meaningfully easier than a topology with no single coordinating point.

Peer-to-Peer Agent Communication

Intuition & Practical Use

Instead of one central coordinator, agents communicate directly with each other, negotiating and passing information peer to peer, with no single agent holding the full plan. This can be more flexible for genuinely decentralized problems where no single agent should (or sensibly could) hold the entire plan up front — but it trades away the orchestrator-worker pattern's single, inspectable coordination point, making the overall system's behavior meaningfully harder to trace and debug when something goes wrong, since there's no one place that ever had the full picture.

Agent Specialization & Role Design

Intuition & Practical Use

Splitting responsibilities across specialized agents (a researcher, a writer, a critic) can genuinely outperform one generalist agent when each role benefits from a distinct prompt, tool set, or even model choice tuned to that specific sub-task — the same reason a human team of specialists can outperform one generalist on a sufficiently complex project. It sometimes doesn't help: if the sub-tasks aren't actually distinct enough to benefit from separate specialization, splitting them just adds coordination overhead without unlocking any real quality gain a well-prompted single agent couldn't already achieve.

Coordination & Hand-Off Mechanics

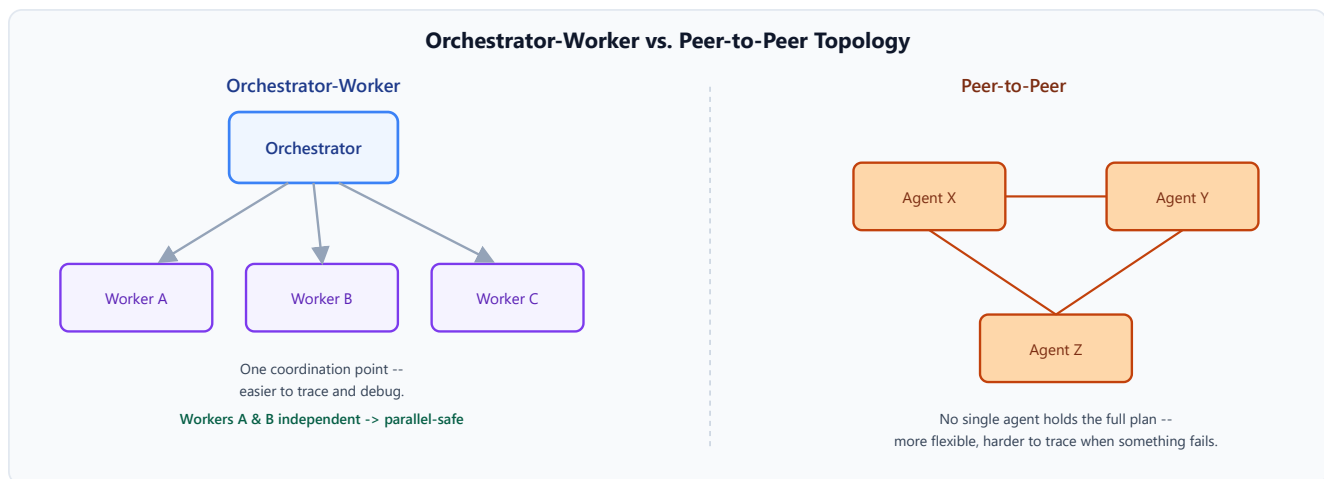
Intuition & Practical Use

The concrete mechanics of getting one agent's output correctly into another agent's input — what format the hand-off takes, whether the receiving agent gets the full context or just a summary, and what happens if the receiving agent decides it needs something the sending agent didn't provide. A hand-off that loses critical context (passing only a final answer with none of the reasoning that produced it) can force the receiving agent to redo work the first agent already did, silently reintroducing the cost/latency the specialization was supposed to save.

Safe Parallel vs. Sequential Agent Execution

Intuition & Practical Use

The exact same dependency-analysis principle Module 02 establishes for individual tool calls applies one level up, to whole agents: independent sub-agents whose tasks don't depend on each other's output — a researcher gathering background facts and a separate agent checking a document's formatting — can genuinely run concurrently, cutting wall-clock time the same way Module 02's parallel tool calls do. Agents in a genuine hand-off relationship — a writer that needs the researcher's findings as its actual input — must run sequentially, for the same reason a tool call needing another tool's result must wait for it. Building the real dependency graph between planned agent invocations, not assuming everything can run at once, is what determines which parts of a multi-agent system can actually be parallelized safely.



When NOT to Use Multi-Agent (Applying Module 01's Framework)

Intuition & Practical Use

Rather than re-deriving a separate argument here, this is Module 01's formal decision framework applied specifically to the single-agent-vs-multi-agent boundary: multi-agent is the highest-complexity, highest-cost, least-controllable option on that table, and it's justified only when the task genuinely decomposes into specialized sub-problems that benefit from separate, focused agents — not merely because splitting the work "sounds" more sophisticated. A single well-designed agent handling a task that doesn't actually decompose cleanly will usually outperform a poorly-decomposed multi-agent system on cost, latency, and reliability all at once, exactly as the decision table predicts.

3. Implementation & Reference Code

Below is a minimal orchestrator-worker coordinator with an explicit dependency graph — determining which workers can run in parallel and which must wait for another worker's output, applying Module 02's dependency-analysis principle at the agent level.

```
from dataclasses import dataclass, field

@dataclass
class WorkerTask:
    name: str
    depends_on: list[str] = field(default_factory=list)

def topological_batches(tasks: list[WorkerTask]) -> list[list[str]]:
    """Groups tasks into sequential batches, where every task within a batch has no
    dependency on any other task in that same batch -- i.e. everything in one batch
    is safe to run in parallel; batches themselves must run in order."""
    remaining = {t.name: set(t.depends_on) for t in tasks}
    completed: set[str] = set()
    batches: list[list[str]] = []

    while remaining:
        ready = [name for name, deps in remaining.items() if deps <= completed]
        if not ready:
            raise ValueError(f"Circular or unresolvable dependency among: {list(remaining)}")
        batches.append(sorted(ready))
        for name in ready:
            completed.add(name)
            remaining[name] -= {name}
```

```

        completed.update(ready)
    for name in ready:
        del remaining[name]
    return batches

@dataclass
class Orchestrator:
    """Minimal orchestrator-worker coordinator: dispatches each batch's workers
    (conceptually in parallel), then assembles results before the next batch."""

    def run(self, tasks: list[WorkerTask], worker_fn) -> dict:
        batches = topological_batches(tasks)
        results: dict = {}
        for batch in batches:
            print(f" Dispatching batch (parallel-safe): {batch}")
            for name in batch:
                results[name] = worker_fn(name, results)
        return results

if __name__ == "__main__":
    # Researcher and formatter are independent (no shared dependency);
    # Writer needs Researcher's output; Critic needs Writer's output.
    tasks = [
        WorkerTask("researcher"),
        WorkerTask("formatter"), # independent of researcher -- safe to run in the same batch
        WorkerTask("writer", depends_on=["researcher"]),
        WorkerTask("critic", depends_on=["writer"]),
    ]

    batches = topological_batches(tasks)
    print(f"Execution batches: {batches}")
    assert batches[0] == ["formatter", "researcher"] # both independent, same (first) batch
    assert batches[1] == ["writer"] # depends on researcher -- next batch
    assert batches[2] == ["critic"] # depends on writer -- final batch

    def fake_worker(name: str, prior_results: dict) -> str:
        if name == "researcher":
            return "findings: X causes Y"
        if name == "formatter":
            return "format: markdown"
        if name == "writer":
            return f"draft based on [{prior_results['researcher']}]"
        if name == "critic":
            return f"review of [{prior_results['writer']}]"
        raise ValueError(name)

    orchestrator = Orchestrator()
    results = orchestrator.run(tasks, fake_worker)
    print(f"\nFinal results: {results}")
    assert "findings" in results["researcher"]
    assert results["researcher"] in results["writer"] # writer's output genuinely used
    researcher's result
    print("\nDependency-aware orchestration verified: independent tasks batched together,
    dependent tasks sequenced correctly.")

    # A circular dependency is correctly rejected, not silently mishandled
    try:
        topological_batches([WorkerTask("a", depends_on=["b"]), WorkerTask("b", depends_on=
["a"])]))

```

```
raise AssertionError("Should have raised on circular dependency")
except ValueError as e:
    print(f"\nCircular dependency correctly rejected: {e}")
```

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Letting genuinely distinct sub-problems within a larger task be handled by separately-specialized agents, each tuned to its own sub-task, when a single generalist agent would underperform on the combination.
- **Why Introduced over Legacy Approaches:** A single agent forced to hold an entire complex, multi-faceted task in one line of reasoning can underperform specialized agents each focused on a narrower, better-tuned sub-task — the same reason specialized human teams often outperform one generalist on sufficiently complex work.
- **Key Failure Modes & Limitations:** Coordination overhead and cost multiplication scale with agent count; a lossy hand-off between agents can force redundant re-work; peer-to-peer topologies with no single coordination point are genuinely harder to trace and debug than orchestrator-worker's single inspectable coordination point.

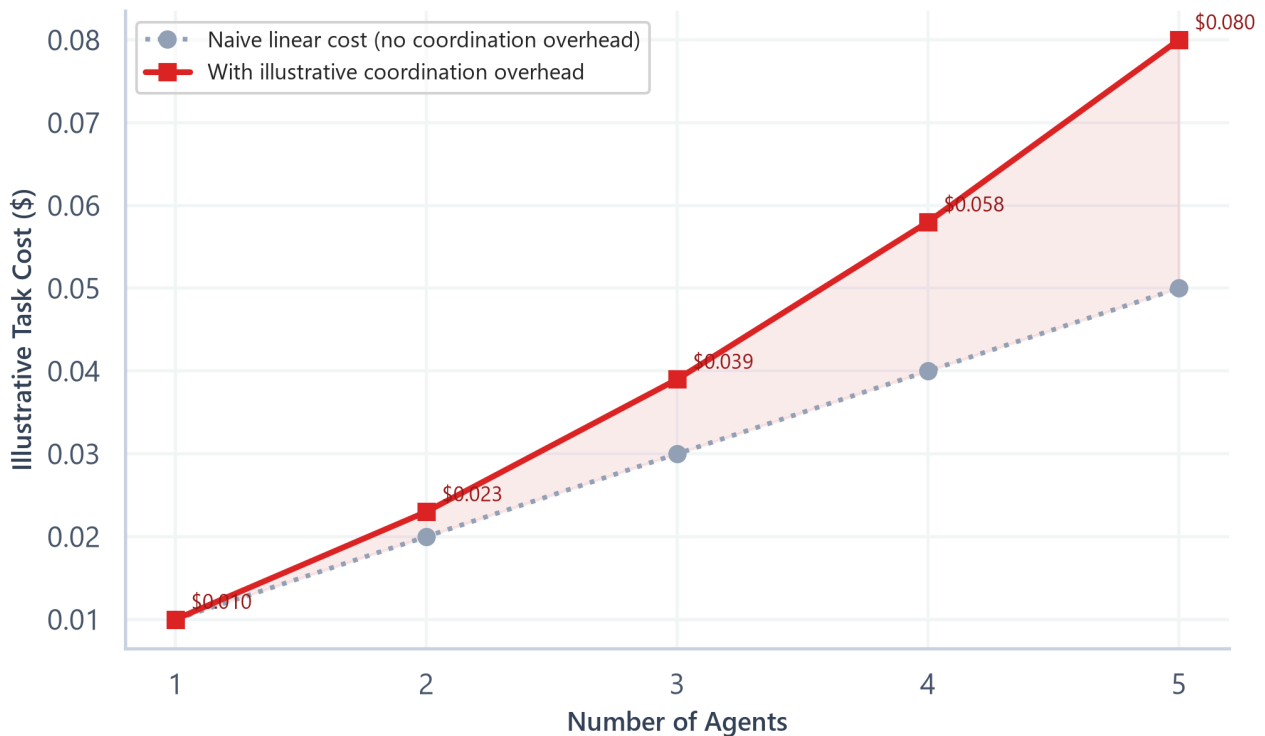
2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Cost and latency scale with the number of agents invoked, reduced by however much genuine independence allows safe parallel execution (this module's dependency-batching mechanism) — worst case is fully sequential (no independence), best case is bounded by the critical path through the real dependency graph.
- **Space/Memory Footprint:** Each agent typically maintains its own context/state (Module 04), so total system memory scales with agent count, not just task complexity.
- **Primary Bottleneck Type:** Coordination-latency-bound when agents must run sequentially due to real dependencies; cost-bound from the direct multiplication of per-agent LLM costs, independent of whether execution is parallelized (parallelizing saves wall-clock time, not total cost).
- **Variable Legend:** No closed-form formula variables, per this module's prose/procedural scope; the illustrative cost-scaling plot below uses invented constants, not a derived formula.

3. Production & Scalability

- **Deployment Considerations:** Build the real dependency graph between planned agent invocations explicitly before assuming anything can run in parallel; prefer orchestrator-worker over peer-to-peer when debuggability and traceability matter more than maximal decentralization flexibility, since the single coordination point is a real operational asset when something goes wrong in production.

ILLUSTRATIVE Multi-Agent Cost Model -- NOT a Measured Production Curve



- **Plot Interpretation:** This is an **illustrative cost model, not a measured production curve** — no notebook in this topic measures a real multi-agent system's cost scaling. The qualitative shape (cost growing faster than linearly with agent count, from an invented coordination-overhead factor) is the point being illustrated: coordination overhead is a real, structural reason multi-agent cost doesn't scale as cheaply as "just add more agents" might naively suggest, even though the specific curve shown here uses invented, not measured, numbers.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: How would you decide between orchestrator-worker and peer-to-peer for a new multi-agent system?

Default to orchestrator-worker for its single, inspectable coordination point and easier debuggability; reach for peer-to-peer specifically when the problem is genuinely decentralized enough that no single agent should sensibly hold the entire plan — not as a default architecture choice.

Question: Two agents both need to read the same shared document as part of their task. Is that a dependency that forces sequential execution?

Not by itself — reading the same input is not a dependency in the sense that matters here; the real question is whether either agent's *output* is needed as the *other's input*. Two agents independently reading the same document and producing independent outputs are still safe to parallelize.

Module 07: Agent Frameworks Landscape

1. Introduction & Intuition

The Core Bottleneck

Every concept in this topic so far — reasoning loops, tool calling, MCP, context/state/memory, durable orchestration, multi-agent coordination — can be built from scratch. Frameworks exist because building all of it correctly, every time, for every new project, is real, repeated engineering effort most teams shouldn't pay for from zero. But a framework is never a neutral, free abstraction: it makes real opinionated choices about how control flow, state, and observability work, and those choices become *your* system's choices the moment you adopt it. The interview-relevant skill isn't memorizing each framework's current API surface — that changes constantly — it's being able to compare them along the dimensions that actually matter for a production decision, and know when the right choice is to build the custom loop instead.

High-Level Intuition

Choosing an agent framework is like choosing a prefabricated building system instead of building from raw materials. It's genuinely faster to get something solid up quickly, and a good prefab system has already solved problems you'd otherwise hit yourself. But you're also now living inside its assumptions about where walls can go — and if your actual requirements don't fit those assumptions, you're either fighting the system or living with a compromise you didn't choose. The right question is never "which prefab system is best," it's "do my requirements fit any prefab system's assumptions well enough to be worth the speed, or do I need to build custom."

2. Core Concepts & Mathematical Formulation

This module is a comparative survey, kept deliberately conceptual rather than API-tutorial — framework APIs change fast enough that memorizing today's syntax has a short shelf life, while the underlying architectural trade-offs are stable and interview-relevant regardless of which specific framework version is current.

Six Dimensions for Comparing Any Agent Framework

Intuition & Practical Use

Rather than a feature-by-feature tour of each framework, every framework in this space can be compared along six stable dimensions — the actual questions worth asking before adopting one:

Dimension	What It Actually Asks
Architecture	Is control flow graph-based, conversational (message-passing between agents), role-based (agents defined by persona/responsibility), or provider-native (built into a specific model provider's own API)?

Dimension	What It Actually Asks
Control	How much explicit control flow does the framework give you (you define every transition) vs. how much does it decide for you (an implicit loop you configure but don't fully see)?
State	How does the framework represent and persist state — is durable checkpointing (Module 05) built in, bolted on, or absent entirely?
Observability	How easy is it to trace a run step by step after the fact — does the framework expose a full execution trace, or does debugging mean adding your own logging inside its abstractions?
Extensibility	How easily do custom tools, custom agents, or custom logic plug into the framework's own abstractions, versus requiring you to work around them?
Lock-In	How much of your code becomes framework-specific and hard to port elsewhere if you later need to switch — is your core logic expressed in the framework's own types/abstractions, or kept mostly independent of it?

The Major Frameworks, Compared Along These Dimensions

Intuition & Practical Use

- **LangGraph** — graph-based architecture (Module 05's explicit nodes/edges/state model is directly what it implements); high explicit control (you define the graph); durable state/checkpointing built in as a first-class feature; strong observability via full graph execution traces; high extensibility (custom nodes are just functions); real lock-in risk since your control flow is expressed directly in its graph abstraction.
- **AutoGen** — conversational multi-agent architecture (agents coordinate by exchanging messages, closer to this topic's peer-to-peer pattern than orchestrator-worker by default); lower explicit control (the conversation pattern drives a lot of the flow implicitly); state/checkpointing support varies by version and is less central to its core model than LangGraph's; observability requires tracing the message exchange itself; extensibility is strong for adding new conversational agents; moderate lock-in, concentrated in its conversation-orchestration abstractions.
- **CrewAI** — role-based multi-agent architecture (agents defined by an explicit persona/role/goal, closer to Module 06's orchestrator-worker pattern by convention); moderate explicit control (roles and tasks are defined explicitly, but execution flow within a role is more implicit); state handling is comparatively lighter-weight than LangGraph's; observability tooling is improving but was historically thinner than graph-based alternatives; extensibility is strong for adding new roles/tasks; lock-in is concentrated in its role/task/crew abstractions.
- **OpenAI's Agents SDK / Assistants API** — provider-native architecture (built directly into a specific model provider's own platform); control and state are largely managed by the provider's own infrastructure rather than your own code, trading control/portability for reduced operational burden; observability is whatever the provider's own tooling exposes; extensibility is bounded by what the provider's API surface supports; the highest lock-in of the four, since the architecture is inseparable from that specific provider's platform.

When to Build a Custom Agent Loop vs. Adopt a Framework

Intuition & Practical Use

Build custom when the task's requirements are narrow and well-understood enough that a framework's general-purpose abstractions add more overhead (learning curve, working around its opinions, lock-in) than they save — Module 01's own reference ReAct loop is a real, complete example of how little code a well-scoped custom loop

actually needs. Adopt a framework when the task genuinely needs several of the capabilities a mature framework has already solved well — durable checkpointing, multi-agent coordination, rich observability — and rebuilding all of that from scratch would cost more engineering time than the framework's opinions cost in flexibility. This is, again, the same complexity-ladder discipline from Module 01: adopt the framework's added machinery only as far as the task's genuine requirements justify it, evaluated against these same six dimensions rather than "which one is popular."

3. Implementation & Reference Code

Framework-specific code is intentionally out of scope here — this module is about comparison criteria, not API tutorials, and framework APIs shift fast enough that concrete code examples would go stale quickly. Below instead is a small, structural comparison helper: a data-driven way to score candidate frameworks against a specific project's actual requirements along the six dimensions above, illustrating the *decision process*, not any one framework's syntax.

```
from dataclasses import dataclass, field

@dataclass
class FrameworkProfile:
    name: str
    # Scored 1 (low) - 5 (high) along each dimension, illustrative ratings for comparison purposes
    control: int
    durable_state: int
    observability: int
    extensibility: int
    lock_in: int # higher = MORE lock-in (worse, for a project wanting portability)

@dataclass
class ProjectRequirements:
    needs_durable_state: bool
    needs_multi_agent: bool
    portability_matters: bool

def score_framework(profile: FrameworkProfile, reqs: ProjectRequirements) -> float:
    """Illustrative scoring: rewards dimensions the project actually needs,
    penalizes lock-in specifically when portability matters to this project."""
    score = profile.control + profile.observability + profile.extensibility
    if reqs.needs_durable_state:
        score += profile.durable_state
    if reqs.portability_matters:
        score -= profile.lock_in
    return score

if __name__ == "__main__":
    langgraph = FrameworkProfile("LangGraph", control=5, durable_state=5, observability=5,
    extensibility=4, lock_in=4)
    provider_native = FrameworkProfile("Provider-Native SDK", control=2, durable_state=3,
    observability=3, extensibility=2, lock_in=5)

    # A project that genuinely needs durable state and cares about portability
    reqs = ProjectRequirements(needs_durable_state=True, needs_multi_agent=False,
    portability_matters=True)

    langgraph_score = score_framework(langgraph, reqs)
```

```

    provider_score = score_framework(provider_native, reqs)
    print(f"LangGraph score for this project's requirements: {langgraph_score}")
    print(f"Provider-native SDK score for this project's requirements: {provider_score}")
    assert langgraph_score > provider_score, "The framework matching this project's real
requirements should score higher"

    # The SAME frameworks, for a project that does NOT care about portability and doesn't need
    durable state
    reqs_different = ProjectRequirements(needs_durable_state=False, needs_multi_agent=False,
portability_matters=False)
    langgraph_score2 = score_framework(langgraph, reqs_different)
    provider_score2 = score_framework(provider_native, reqs_different)
    print(f"\nSame frameworks, different requirements:")
    print(f"LangGraph score: {langgraph_score2}")
    print(f"Provider-native SDK score: {provider_score2}")
    print("\nVerified: the 'right' framework depends entirely on the specific project's
requirements, not a fixed ranking.")

```

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Avoiding re-solving already-solved problems (durable state, multi-agent coordination, observability tooling) from scratch on every new agent project, at the cost of adopting a framework's specific opinions about how those problems should be solved.
- **Why Introduced over Legacy Approaches:** Hand-rolled agent loops before frameworks matured meant every team independently reinvented checkpointing, tool-calling glue, and tracing — often incompletely or inconsistently; mature frameworks concentrate that engineering effort into a shared, battle-tested layer.
- **Key Failure Modes & Limitations:** Adopting a framework whose opinions don't actually fit the task's real requirements creates friction that outweighs the time saved; heavy lock-in makes migrating away from a framework choice expensive if requirements change substantially later; thin observability in some frameworks pushes debugging effort back onto the team anyway, eroding part of the framework's promised value.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Not a framework-level concern — the underlying LLM calls and tool executions cost the same regardless of which orchestration layer wraps them; framework choice affects engineering velocity and observability, not model-level compute cost.
- **Space/Memory Footprint:** Varies by framework's state-management approach — a framework with first-class durable checkpointing (Module 05) has a real, explicit storage footprint for that state; a thinner framework may leave state management (and its footprint) entirely up to the implementing team.
- **Primary Bottleneck Type:** Not a runtime bottleneck — the real cost here is engineering/organizational: learning curve, lock-in risk, and the gap between a framework's opinions and the project's actual needs.
- **Variable Legend:** No closed-form formula variables, per this module's prose/comparative scope; the reference code's dimension scores are illustrative ratings for comparison purposes, not measured benchmarks.

3. Production & Scalability

- **Deployment Considerations:** Re-evaluate the six-dimension comparison against the *project's* actual requirements, not a generic "which framework is best" ranking — the same framework can be the clear right

choice for one project and clear overkill (or a poor fit) for another, exactly as the reference code's scoring example demonstrates with the same two frameworks scoring oppositely under different requirements.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: Your team has already built significant custom logic on a hand-rolled agent loop. When would switching to a framework actually be worth the migration cost?

When the custom loop is now solving problems (durable state at scale, multi-agent coordination, production-grade observability) that a mature framework already solves well, and the ongoing cost of maintaining that custom machinery exceeds the one-time migration cost plus the framework's lock-in risk — not simply because a framework exists.

Question: How would you evaluate lock-in risk concretely before committing to a framework?

Look at how much of your core business logic would need to be expressed directly in the framework's own types/abstractions (its graph nodes, its role definitions) versus how much can stay as plain, portable code that the framework merely orchestrates — the more logic lives inside framework-specific abstractions, the higher the real cost of migrating away later.

Module 08: Agent Evaluation & Debugging

1. Introduction & Intuition

The Core Bottleneck

Evaluating a single-turn LLM call is comparatively simple: one input, one output, score it. Evaluating an agent is a fundamentally different problem — the *final answer* might be correct even though the path that produced it was wasteful, or worse, correct by luck despite a genuinely wrong tool call along the way that happened not to matter this time. Judging only the final output misses exactly the information needed to actually improve the system: which step, if any, went wrong, and whether it went wrong in a way that's likely to recur. Agent evaluation has to look at the whole trajectory, not just where it ended up.

High-Level Intuition

Grading a student only on their final exam answer tells you whether they got the right number, but not whether they used the right method and got lucky with rounding, or used a genuinely sound method and made one recoverable arithmetic slip. Grading their full worked solution — every step — tells you which one actually happened, and that distinction is exactly what determines whether you'd trust them to get the next, slightly different problem right too. Agent trajectory evaluation is grading the full worked solution, not just the final number.

2. Core Concepts & Mathematical Formulation

Full Trajectory Evaluation vs. Final-Output-Only Evaluation

Intuition & Practical Use

An agent's full trajectory is every Thought/Action/Observation step (Module 01) it actually took, not just the answer it eventually returned. Evaluating only the final output can't distinguish "the agent reasoned soundly and got the right answer" from "the agent made an error that happened not to matter this time" — and that distinction matters enormously for predicting whether the *next*, slightly different query will also come out right. Full trajectory evaluation looks at every step, which is exactly what makes the seven metrics below possible to compute in the first place.

LLM-as-Judge for Agent Trajectories, and Its Pitfalls

Intuition & Practical Use

An LLM can be prompted to review a full trajectory and judge whether each step was reasonable, whether the tool choices were correct, and whether the final answer is genuinely supported by what was actually found along the way — a scalable alternative to exhaustive human review. The pitfalls are the same ones any LLM-as-judge approach carries (prompt sensitivity, judge bias) plus one specific to agents: a judge reviewing a long, multi-step trajectory has more surface area to be inconsistent across than a judge reviewing one single-turn response, so trajectory-level LLM-as-judge scoring needs the same discipline as any LLM-based evaluation — a fixed, stable judging rubric, tracked as a trend over time, not compared across differently-worded judge prompts.

Agent Observability & Tracing

Intuition & Practical Use

The practical infrastructure that makes trajectory evaluation and debugging possible in production at all: every Thought, Action, Observation, tool call (with arguments and results), and their timestamps/latencies logged for every run, queryable after the fact (Langfuse-style tracing is the concrete tool matching the production stack this candidate already uses). Without this, "why did this specific run go wrong" requires trying to reproduce the exact conditions after the fact — often impossible, since a live external tool's response or a model's own non-determinism may not reproduce the same way twice.

Common Agent Failure Modes

Intuition & Practical Use

A short, recurring catalog worth recognizing by pattern, not just definition: **infinite/repeating loops** (the agent keeps taking the same or a functionally-equivalent action without making progress — exactly what Module 01's `max_steps` guard exists to bound, but bounding it doesn't diagnose *why* it happened); **tool misuse** (the right tool called with wrong arguments, or a tool called in a context it was never meant for); **planning failures** (a plan-and-execute agent, per Module 01, commits to a plan that turns out to be wrong and doesn't adequately recover); **premature termination** (the agent decides it has "enough information" to answer when it genuinely doesn't, a failure of the `is_sufficient` judgment from Module 01's own reference loop).

Hand Calculation: Agent Trajectory Metrics for One Toy Task

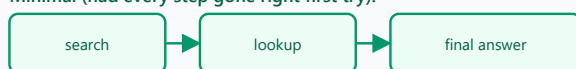
One toy 5-step agent trajectory, plus a toy batch of 5 total tasks (4 succeeded, 1 failed) for the two batch-level metrics.

The one toy trajectory (5 steps):

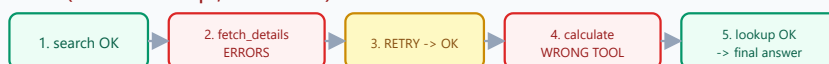
1. Correct tool (`search`) chosen, executes successfully.
2. Correct tool (`fetch_details`) chosen, but the call itself **errors** (e.g., a timeout) — a tool failure, independent of tool selection.
3. **Retry** of step 2's tool call — succeeds this time.
4. **Wrong** tool chosen (`calculate` instead of `lookup`) — a tool-selection error; it executes without erroring, but returns an unhelpful result.
5. Correct tool (`lookup`) chosen after replanning, executes successfully — the agent now has enough information and produces its final answer.

Minimal Path (3 steps) vs. Actual Trajectory (5 steps)

Minimal (had every step gone right first try):



Actual (this run -- 5 steps, real detours):



Trajectory Efficiency = 3 minimal / 5 actual = 0.6 -- the 2 extra steps are exactly the retry (step 3) and the wrong-tool detour (step 4).

- **Trajectory Efficiency.** The minimal path (had the agent picked correctly on the first try at every step) would have been 3 steps — `search` → `lookup` → final answer:

$$\text{Trajectory Efficiency} = \frac{\text{Steps}_{\text{minimal}}}{\text{Steps}_{\text{actual}}} = \frac{3}{5} = 0.6$$

- **Tool-Selection Accuracy.** Of the 5 tool-call attempts, 4 chose the objectively correct tool for that step (steps 1, 2, 3, 5 — step 3's retry still chose the same, correct tool); only step 4 chose the wrong one:

$$\text{Tool-Selection Accuracy} = \frac{4}{5} = 0.8$$

- **Tool Failure Rate.** Of the 5 tool-call attempts, exactly 1 (step 2) errored on execution, independent of whether the tool choice itself was correct:

$$\text{Tool Failure Rate} = \frac{1}{5} = 0.2$$

- **Retry Rate.** Of the 5 total steps, exactly 1 (step 3) was a retry of a prior step:

$$\text{Retry Rate} = \frac{1}{5} = 0.2$$

The toy 5-task batch (Task 1 is the trajectory above at 5 steps/succeeded; Task 2: 3 steps/succeeded; Task 3: 4 steps/succeeded; Task 4: 6 steps/**failed**; Task 5: 3 steps/succeeded):

- **Task Success Rate.**

$$\text{Task Success Rate} = \frac{N_{\text{successful}}}{N_{\text{total}}} = \frac{4}{5} = 0.8$$

- **Steps per Successful Task** (Task 4 excluded — it failed):

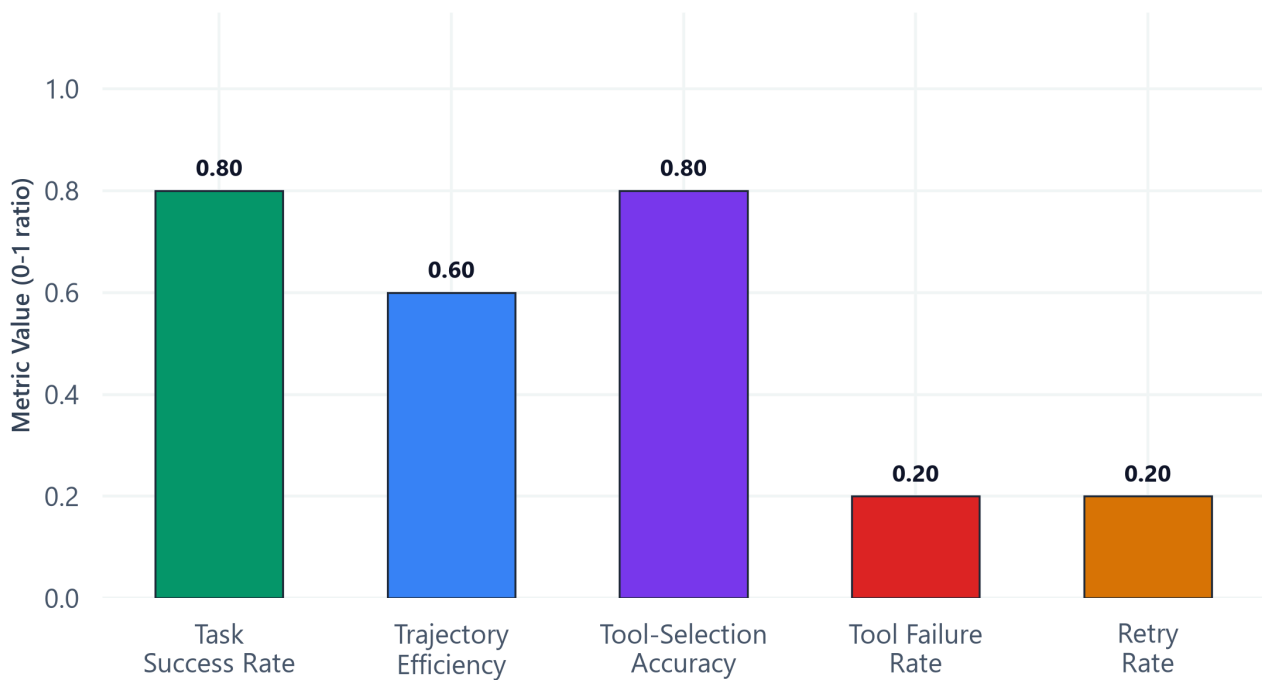
$$\text{Steps per Successful Task} = \frac{5 + 3 + 4 + 3}{4} = \frac{15}{4} = 3.75$$

- **Cost per Successful Task** (illustrative per-task costs from Module 02's cost model — \\$.012, \\$.007, \\$.009, \\$.006 for the 4 successful tasks; Task 4's cost is excluded, the same way its steps were):

$$\text{Cost per Successful Task} = \frac{0.012 + 0.007 + 0.009 + 0.006}{4} = \frac{\$0.034}{4} = \$0.0085$$

Seven metrics, from the same two toy examples, telling seven genuinely different — and here, deliberately non-redundant — stories: efficiency (0.6) says the agent took real, avoidable extra steps; tool-selection accuracy (0.8) and tool failure rate (0.2) together show that *most* of that inefficiency came from one genuine tool-selection mistake (step 4), not primarily from unreliable tools; retry rate (0.2) shows the one tool failure was successfully recovered from; and the batch-level success rate (0.8), steps-per-success (3.75), and cost-per-success (\$.0085) show what all of this costs in aggregate once tasks that never succeeded at all are accounted for separately.

Agent Trajectory Metrics: One Toy 5-Step Trajectory + a Toy 5-Task Batch



- **Plot Interpretation:** The five 0-1 ratio metrics plotted together make the diagnosis visually immediate — a high tool-selection accuracy (0.8) next to a lower trajectory efficiency (0.6) is exactly the visual signature of "the agent mostly chose correctly, but paid a real efficiency cost from the one time it didn't, plus recovering from one tool failure." Steps-per-successful-task (3.75) and cost-per-successful-task (\\$0.0085) are reported as text rather than on this chart, since they're absolute-unit metrics, not 0-1 ratios, and don't share a meaningful scale with the other five.

3. Implementation & Reference Code

Below is a self-contained implementation of all seven hand-calculated metrics, computing them directly from a structured trajectory log — matching the worked example above exactly.

```
from dataclasses import dataclass, field
from enum import Enum

class StepOutcome(Enum):
    CORRECT_TOOL_SUCCESS = "correct_tool_success"
    CORRECT_TOOL_ERROR = "correct_tool_error"          # right tool, but the call itself failed
    RETRY_SUCCESS = "retry_success"                    # a retry of a prior failed step, succeeded
    WRONG_TOOL = "wrong_tool"                          # tool-selection error

@dataclass
class TrajectoryStep:
    outcome: StepOutcome

@dataclass
class TaskResult:
```

```

steps: list[TrajectoryStep]
succeeded: bool
cost: float

def trajectory_efficiency(steps_minimal: int, steps_actual: int) -> float:
    return steps_minimal / steps_actual

def tool_selection_accuracy(steps: list[TrajectoryStep]) -> float:
    correct = sum(1 for s in steps if s.outcome != StepOutcome.WRONG_TOOL)
    return correct / len(steps)

def tool_failure_rate(steps: list[TrajectoryStep]) -> float:
    failed = sum(1 for s in steps if s.outcome == StepOutcome.CORRECT_TOOL_ERROR)
    return failed / len(steps)

def retry_rate(steps: list[TrajectoryStep]) -> float:
    retried = sum(1 for s in steps if s.outcome == StepOutcome.RETRY_SUCCESS)
    return retried / len(steps)

def task_success_rate(tasks: list[TaskResult]) -> float:
    return sum(1 for t in tasks if t.succeeded) / len(tasks)

def steps_per_successful_task(tasks: list[TaskResult]) -> float:
    successful = [t for t in tasks if t.succeeded]
    return sum(len(t.steps) for t in successful) / len(successful)

def cost_per_successful_task(tasks: list[TaskResult]) -> float:
    successful = [t for t in tasks if t.succeeded]
    return sum(t.cost for t in successful) / len(successful)

if __name__ == "__main__":
    # The one toy 5-step trajectory
    trajectory = [
        TrajectoryStep(StepOutcome.CORRECT_TOOL_SUCCESS), # 1: search, correct, succeeds
        TrajectoryStep(StepOutcome.CORRECT_TOOL_ERROR),   # 2: fetch_details, correct tool,
errors
        TrajectoryStep(StepOutcome.RETRY_SUCCESS),         # 3: retry of step 2, succeeds
        TrajectoryStep(StepOutcome.WRONG_TOOL),            # 4: calculate (wrong), should have
been lookup
        TrajectoryStep(StepOutcome.CORRECT_TOOL_SUCCESS), # 5: lookup, correct, succeeds -> final
answer
    ]

    eff = trajectory_efficiency(steps_minimal=3, steps_actual=len(trajectory))
    tsa = tool_selection_accuracy(trajectory)
    tfr = tool_failure_rate(trajectory)
    rr = retry_rate(trajectory)

    print(f"Trajectory Efficiency: {eff:.2f}")
    print(f"Tool-Selection Accuracy: {tsa:.2f}")
    print(f"Tool Failure Rate: {tfr:.2f}")
    print(f"Retry Rate: {rr:.2f}")
    assert eff == 0.6 and tsa == 0.8 and tfr == 0.2 and rr == 0.2

```

```

# The toy 5-task batch
tasks = [
    TaskResult(steps=trajectory, succeeded=True, cost=0.012),          # Task 1 (the
trajectory above)
    TaskResult(steps=[TrajectoryStep(StepOutcome.CORRECT_TOOL_SUCCESS)] * 3, succeeded=True,
cost=0.007),
    TaskResult(steps=[TrajectoryStep(StepOutcome.CORRECT_TOOL_SUCCESS)] * 4, succeeded=True,
cost=0.009),
    TaskResult(steps=[TrajectoryStep(StepOutcome.WRONG_TOOL)] * 6, succeeded=False,
cost=0.011), # Task 4: failed
    TaskResult(steps=[TrajectoryStep(StepOutcome.CORRECT_TOOL_SUCCESS)] * 3, succeeded=True,
cost=0.006),
]

tsr = task_success_rate(tasks)
spst = steps_per_successful_task(tasks)
cpst = cost_per_successful_task(tasks)

print(f"\nTask Success Rate: {tsr:.2f}")
print(f"Steps per Successful Task: {spst:.2f}")
print(f"Cost per Successful Task: ${cpst:.4f}")
assert tsr == 0.8
assert spst == 3.75
assert abs(cpst - 0.0085) < 1e-9
print("\nAll seven trajectory/batch metrics verified against the hand calculation.")

```

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Measuring agent quality in a way that can actually distinguish *why* a run succeeded or failed — which step, and what kind of failure — rather than a single pass/fail judgment on the final output alone.
- **Why Introduced over Legacy Approaches:** Single-turn LLM evaluation metrics say nothing about multi-step trajectories at all; judging only an agent's final answer conflates "sound reasoning, correct answer" with "flawed reasoning that got lucky," which are very different signals for predicting future reliability.
- **Key Failure Modes & Limitations:** Full-trajectory evaluation costs proportionally more than final-output-only evaluation (every step needs review, not just the last one); LLM-as-judge trajectory scoring inherits the general prompt-sensitivity/judge-bias pitfalls, with more surface area for judge inconsistency across a long trajectory than a single-turn response.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Evaluation cost scales with the number of steps actually taken across all evaluated trajectories, not just the number of tasks — a real, direct multiplier over single-turn evaluation cost.
- **Space/Memory Footprint:** Full trajectory logging (Module 08's own observability requirement) accumulates storage proportional to step count across all runs, the evaluation-side analog of Module 05's checkpoint-storage growth concern.
- **Primary Bottleneck Type:** Evaluation-cost-bound at full-trajectory granularity for offline evaluation runs; production observability logging itself must stay cheap enough per-step to not add meaningful latency to the live agent run it's tracing.

- **Variable Legend:** $N_{\text{successful}}/N_{\text{total}}$ = successful/total task counts, $\text{Steps}_{\text{minimal}}/\text{Steps}_{\text{actual}}$ = the ideal vs. actual step count for one trajectory, $\text{Cost}_{\text{task}}$ = Module 02's per-task cost model.

3. Production & Scalability

- **Deployment Considerations:** Log full trajectories (every Thought/Action/Observation, with tool arguments/results and timestamps) from day one, the agent-specific analog of `03_advanced_rag` Module 09's retrieval-observability discipline; track these seven metrics as trends over time on a fixed evaluation set, not one-off snapshots, so a regression is caught as a trend rather than discovered only after users notice.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: An agent's task success rate looks fine, but users are complaining about high latency and cost. Which of these seven metrics would you check first?

Steps per successful task and trajectory efficiency — a fine success rate can coexist with agents taking far more steps than necessary to get there, which is exactly the signal these two metrics are designed to surface that success rate alone hides.

Question: How would you distinguish a tool-selection problem from a tool-reliability problem using these metrics?

Tool-selection accuracy and tool failure rate are deliberately separate metrics for exactly this reason — a low tool-selection accuracy points at the model's decision-making (schema/prompt issue, Module 02), while a high tool failure rate with good selection accuracy points at the tool's own reliability (an infrastructure/API issue), and conflating the two into one "things went wrong" number would point engineering effort at the wrong fix.

Module 09: Production Agent Systems, Safety & Security

1. Introduction & Intuition

The Core Bottleneck

A bad output from a single-turn LLM call is bad text — annoying, sometimes costly, but rarely irreversible on its own. A bad decision from an agent with real tool access is a bad *action* — a sent email, a deleted record, a payment charged — and once it's executed, it's executed. This is the qualitative shift production agent systems introduce: the blast radius of a mistake is no longer bounded by "the user reads bad text and (hopefully) doesn't act on it," it's bounded by whatever the agent's tools are actually capable of doing in the real world. Every mechanism in this module exists to keep that blast radius bounded and intentional, not to make the agent smarter.

High-Level Intuition

Giving an intern read-only access to a shared document is low-risk even if they make a mistake reading it. Giving that same intern the keys to the production database, unsupervised, on day one, is a fundamentally different risk — not because the intern got worse, but because the *consequences of a mistake* changed entirely. Production agent safety is the discipline of scoping what an agent can actually do to match how much you actually trust its judgment for that specific action, the same way you wouldn't hand an intern unsupervised production database access before they'd earned that trust — and building in a way to catch and undo mistakes when they happen anyway, because they will.

2. Core Concepts & Mathematical Formulation

This module stays architectural/procedural throughout — production hardening and security policy are structural concerns, consistent with how `03_advanced_rag` Module 09 treated its own production-hardening content.

Guardrails & Permission Boundaries

Intuition & Practical Use

Not every action an agent decides on should execute immediately and unsupervised. A guardrail policy classifies actions into at least two tiers: fully autonomous (the agent executes without waiting) and approval-gated (the agent proposes the action and waits for explicit confirmation before it executes) — the production policy layer built directly on Module 02's confirmation-gate mechanics and Module 05's human-in-the-loop interrupt capability. The classification itself is the real design decision: a read-only lookup is a reasonable candidate for full autonomy; an irreversible, high-consequence action (deleting data, sending a payment, publishing something publicly) is a reasonable candidate for a mandatory approval gate, regardless of how confident the agent's own reasoning appears.

Rate Limiting & Cost/Step Budgets

Intuition & Practical Use

The production enforcement of Module 02's per-task cost model and Module 01's step-count termination guard: a hard ceiling on how many steps, how much cost, or how much real-world action an agent can take within a bounded

window, regardless of what its own reasoning decides. This exists specifically to bound the agent's *own* potential for runaway behavior — a genuinely malformed or adversarial input that causes the agent to loop or escalate shouldn't be able to consume unbounded cost or take unbounded real-world action just because no single step's reasoning looked obviously wrong in isolation.

Indirect Prompt Injection

Intuition & Practical Use

Direct prompt injection is a malicious instruction from the user themselves, trying to manipulate the model directly. **Indirect** prompt injection is different and, for a tool-using agent, often more dangerous: malicious instructions arrive not from the user, but embedded in content the agent processes as *data* and ends up treating as *instructions* — because the agent has no structural way to distinguish "text I'm supposed to read" from "text I'm supposed to obey" once both are sitting in the same context window. Four real sources this shows up from in practice:

- **Tool outputs** — a fetched webpage or API response crafted to contain hidden instructions.
- **Retrieved content** — a poisoned document surfaced by RAG-based retrieval (a risk `03_advanced_rag` covers on the retrieval side; this module covers the same underlying risk from the agent-tool-execution side, not a duplicate treatment).
- **Files** — an uploaded or read document with hidden instructions embedded in its text.
- **External APIs** — a third-party response deliberately crafted to hijack the agent's next action.

The common thread across all four: the agent's context window doesn't inherently distinguish trusted instructions from untrusted data once they're both just text in the same prompt — which is exactly why the mitigations below don't rely on the model "just knowing better."

Layered Mitigation: Isolation, Validation, Least Privilege, Approval Gates, Auditing

Intuition & Practical Use

No single mitigation is a complete defense against indirect prompt injection — production security here is a layered set of controls, each catching what the others might miss:

- **Isolation** — sandboxing tool/content execution so untrusted content can't directly act, only be read (the subject of the next subsection in depth).
- **Validation** — checking or sanitizing untrusted content before it enters context, reducing the odds a hidden instruction ever reaches the model in an interpretable form.
- **Least-privilege tool access** — scoping each agent/tool to the minimum permissions it genuinely needs, so *even a successful* injection has a small blast radius, because the compromised agent still can't do anything beyond its narrow, already-limited grant.
- **Approval gates** — requiring human confirmation before a sensitive action executes, the same Module 02 confirmation-gate mechanic now applied specifically as a security control against an agent that's been manipulated into proposing a harmful action.
- **Auditing** — recording every action taken, so a successful injection that does slip through is at minimum detectable and traceable after the fact, not silently invisible.

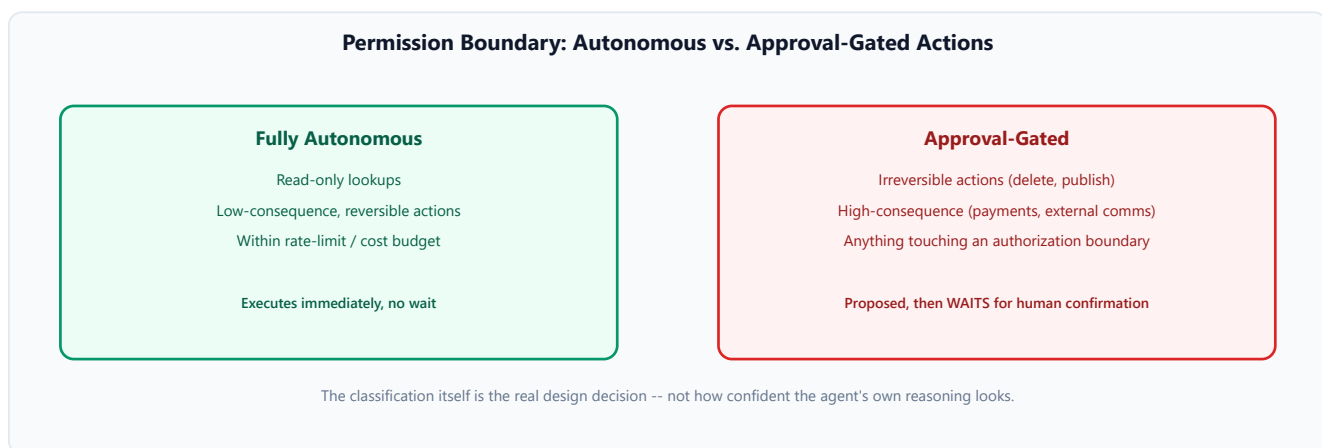
None of these mitigations depend on training the model to be better at recognizing injected instructions — they depend on structurally limiting what damage is possible even when the model gets fooled, which is a fundamentally

more robust posture than trying to make the model injection-proof.

Authorization Boundaries & Sandboxing

Intuition & Practical Use

Authorization boundaries are the structural mechanism least-privilege access is actually built on: per-agent, per-tool, and per-action permission scoping, enforced at the point of execution, not just documented as a policy nobody checks. Sandboxing takes this further — isolating the *environment* a tool executes in (a restricted filesystem, no network access, a disposable container) so that even a tool call the agent was never supposed to make, executing with more permission than intended due to a bug or a successful injection, is still contained by the environment it's running in, not just by the permission check that was supposed to prevent it in the first place. Defense in depth: the permission check is supposed to prevent the bad action; the sandbox is what limits the damage if the permission check somehow fails.



Deployment Patterns: Synchronous vs. Long-Running Async Agents

Intuition & Practical Use

A synchronous request/response agent runs within one HTTP request's lifetime — the caller waits, gets a result, done; simple to reason about, but bounded by whatever timeout the request path can tolerate. A long-running async/background agent (Module 05's durable-execution machinery is exactly what makes this viable) can run for far longer, checkpointing its progress and notifying the caller (or waiting for approval) asynchronously — necessary for genuinely long tasks, but it also means the agent's actions are happening in the background, unsupervised in real time, for however long the task runs, which is exactly why guardrails and approval gates matter *more*, not less, for this deployment pattern.

3. Implementation & Reference Code

Below is a minimal guardrail policy engine — classifying proposed actions into autonomous vs. approval-gated tiers, enforcing a rate/cost budget, and logging every decision to an audit trail, illustrating the layered-mitigation mechanics above.

```
from dataclasses import dataclass, field
from enum import Enum
from datetime import datetime
```

```

class ActionTier(Enum):
    AUTONOMOUS = "autonomous"
    APPROVAL_GATED = "approval_gated"
    BLOCKED = "blocked" # exceeds this agent's authorization boundary entirely

@dataclass
class ProposedAction:
    tool_name: str
    is_reversible: bool
    estimated_cost: float

@dataclass
class AuditEntry:
    timestamp: str
    tool_name: str
    tier: ActionTier
    executed: bool

@dataclass
class GuardrailPolicy:
    """Classifies proposed actions and enforces a hard budget -- the production
    policy layer built on Module 02's confirmation-gate and Module 01's step-guard ideas."""
    authorized_tools: set[str]
    max_total_cost: float
    audit_log: list[AuditEntry] = field(default_factory=list)
    spent: float = 0.0

    def classify(self, action: ProposedAction) -> ActionTier:
        if action.tool_name not in self.authorized_tools:
            return ActionTier.BLOCKED # least-privilege: never authorized for this agent at all
        if not action.is_reversible:
            return ActionTier.APPROVAL_GATED # irreversible -> always gated, regardless of
"confidence"
            return ActionTier.AUTONOMOUS

    def try_execute(self, action: ProposedAction, execute_fn, approve_fn=None) -> str:
        tier = self.classify(action)

        if tier == ActionTier.BLOCKED:
            self._log(action, tier, executed=False)
            raise PermissionError(f"'{action.tool_name}' is outside this agent's authorization
boundary")

        if self.spent + action.estimated_cost > self.max_total_cost:
            self._log(action, tier, executed=False)
            raise RuntimeError(f"Budget exceeded: ${self.spent + action.estimated_cost:.4f} >
${self.max_total_cost:.4f}")

        if tier == ActionTier.APPROVAL_GATED:
            if approve_fn is None or not approve_fn(action):
                self._log(action, tier, executed=False)
                return "[action proposed, awaiting/denied approval -- NOT executed]"

        result = execute_fn(action)
        self.spent += action.estimated_cost
        self._log(action, tier, executed=True)
        return result

```



```

def _log(self, action: ProposedAction, tier: ActionTier, executed: bool) -> None:
    self.audit_log.append(AuditEntry(datetime.now().isoformat(), action.tool_name, tier,
executed))

if __name__ == "__main__":
    policy = GuardrailPolicy(authorized_tools={"search_docs", "send_email"}, max_total_cost=0.05)

    # Autonomous, reversible, authorized -> executes immediately
    lookup = ProposedAction(tool_name="search_docs", is_reversible=True, estimated_cost=0.01)
    result = policy.try_execute(lookup, execute_fn=lambda a: "search results...")
    print(f"search_docs result: {result}")
    assert policy.audit_log[-1].tier == ActionTier.AUTONOMOUS and policy.audit_log[-1].executed

    # Irreversible, authorized -> approval-gated, denied
    send_email = ProposedAction(tool_name="send_email", is_reversible=False, estimated_cost=0.01)
    denied = policy.try_execute(send_email, execute_fn=lambda a: "email sent", approve_fn=lambda
a: False)
    print(f"send_email (denied): {denied}")
    assert policy.audit_log[-1].tier == ActionTier.APPROVAL_GATED and not
policy.audit_log[-1].executed

    # Irreversible, authorized -> approval-gated, approved this time
    approved = policy.try_execute(send_email, execute_fn=lambda a: "email sent", approve_fn=lambda
a: True)
    print(f"send_email (approved): {approved}")
    assert policy.audit_log[-1].executed

    # Outside authorization boundary entirely -> blocked, never even reaches approval
    delete_action = ProposedAction(tool_name="delete_database", is_reversible=False,
estimated_cost=0.0)
    try:
        policy.try_execute(delete_action, execute_fn=lambda a: "deleted")
        raise AssertionError("Should have been blocked")
    except PermissionError as e:
        print(f"\nBlocked (outside authorization boundary): {e}")
        assert policy.audit_log[-1].tier == ActionTier.BLOCKED

    print(f"\nFull audit trail ({len(policy.audit_log)} entries):")
    for entry in policy.audit_log:
        print(f" [{entry.tier.value}] {entry.tool_name} -> executed={entry.executed}")
    print("\nGuardrail policy verified: least-privilege blocking, approval gating, and audit
logging all functioning correctly.")

```

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Bounding the real-world blast radius of an agent's mistakes — a category of risk that doesn't exist for a text-only LLM call, since an agent with tool access can take irreversible real actions, not just generate bad text.
- **Why Introduced over Legacy Approaches:** Trusting the model's own reasoning as the sole safety mechanism has no structural defense against a confidently-wrong decision or a successful indirect prompt injection; guardrails, budgets, and layered security controls provide a defense that holds even when the model's own judgment fails.

- **Key Failure Modes & Limitations:** Misclassifying an irreversible action as autonomous removes the one safety check that would have caught a bad decision before it executed; relying on a single mitigation (e.g., only least-privilege, with no auditing) leaves a real gap if that one control fails; rate/cost budgets that are too loose don't actually bound worst-case behavior.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Not compute-bound — guardrail classification and budget checks are cheap, constant-time operations relative to the LLM call or tool execution they gate.
- **Space/Memory Footprint:** Audit logs accumulate proportionally to total agent action volume across the system's lifetime — a real, monotonically-growing storage cost requiring its own retention policy, the security-specific analog of Module 04's unbounded-memory-growth concern.
- **Primary Bottleneck Type:** Not a runtime bottleneck — the real cost is engineering discipline: correctly classifying every action's risk tier, and correctly scoping every tool's authorized permission set, neither of which a system can verify automatically without deliberate, ongoing review.
- **Variable Legend:** No closed-form formula variables, per this module's prose/procedural scope.

3. Production & Scalability

- **Deployment Considerations:** Default to the most restrictive tier (approval-gated, or even blocked) for any newly-added tool or action type until it's been deliberately reviewed and reclassified, rather than defaulting to autonomous and hoping nothing goes wrong; treat audit logs as a first-class production artifact from day one, since they're what makes a successful injection or a bad decision detectable and traceable after the fact rather than silently invisible.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: How would you defend against indirect prompt injection from a document your RAG pipeline retrieves?

Layer the defenses rather than relying on one: validate/sanitize retrieved content before it enters context, ensure the agent's tools operating on that content carry least-privilege access so a successful injection has limited reach, gate any consequential action behind human approval, and audit-log everything so a slip-through is at least detectable — the same layered approach this module applies generally, with the retrieval-specific detection/filtering itself covered on `03advancedrag``'s side.

Question: Why is a long-running async agent a bigger safety concern than a synchronous one, even with identical tools and permissions?

A synchronous agent's actions happen within one bounded request the caller is actively waiting on; a long-running async agent's actions happen in the background, unsupervised in real time, for however long the task runs — the same guardrails matter more, not less, because there's a longer real-world window for something to go wrong before anyone's actively watching.